

Explainable Robotic Plan Execution Monitoring Under Partial Observability

Gokay Coruhlu^{ID}, Esra Erdem, *Member, IEEE*, and Volkan Patoglu^{ID}, *Member, IEEE*

Abstract—Successful plan generation for autonomous systems is necessary but not sufficient to guarantee reaching a goal state by an execution of a plan. Various discrepancies between an expected state and the observed state may occur during the plan execution (e.g., due to unexpected exogenous events, changes in the goals, or failure of robot parts) and these discrepancies may lead to plan failures. For that reason, autonomous systems should be equipped with execution monitoring algorithms so that they can autonomously recover from such discrepancies. We introduce a plan execution monitoring algorithm that operates under partial observability. This algorithm relies on novel formal methods for hybrid prediction, diagnosis and explanation generation, and planning. The prediction module generates an expected state after the execution of a part of the plan from an incomplete state to check for discrepancies. The diagnostic reasoning module generates meaningful hypotheses to explain failures of robot parts. Unlike the existing diagnosis methods, the previous hypotheses can be revised, based on new partial observations, increasing the accuracy of explanations as further information becomes available. The replanning module considers these explanations while computing a new plan that would avoid such failures. All these reasoning modules are hybrid in that they combine high-level logical reasoning with low-level feasibility checks based on probabilistic methods. We experimentally show that these hybrid formal reasoning modules improve the performance of plan execution monitoring.

Index Terms—Diagnostic reasoning, hybrid planning, partial observability, plan execution monitoring, replanning with guidance.

I. INTRODUCTION

SUCCESSFUL deployment of robotic assistants in real-world environments necessitates these systems to be endowed with high-level cognitive functions that allow them to robustly deal with high complexity and wide variability of their surroundings. For that reason, autonomous robot systems are equipped with the ability to plan their actions to reach their goals. Independent of the underlying planning paradigm and/or algorithm, planning requires models of the environment and the robot to produce a valid plan. However, due to uncertainties

and surprises, the world may evolve differently from the expected (as described by the model) during the execution of a plan. Therefore, robust execution of plans requires 1) detecting discrepancies between the expected and the observed states that affect the validity of the rest of the plan (such discrepancies are called relevant), 2) diagnosing the cause of and generating explanations for these discrepancies, and 3) implementing a means of recovery. The process of verification of the continued validity of an executed plan, and implementation of a means of recovery as needed, is known as *execution monitoring* [1], [2].

Three main reasoning tasks take place during plan execution monitoring: *prediction*, *diagnosis/explanation generation*, and *replanning*. In particular, to be able to detect discrepancies, the expected state after the execution of the relevant part of the plan needs to be predicted, such that it can be compared with the current observed state. Once a discrepancy is detected, diagnostic reasoning is required to determine its cause, as it may have occurred early in the plan execution and may not have been easily detected by sensors. Finally, once the cause of the discrepancy is determined, a new plan needs to be generated by taking its explanation into account. Note that diagnostic reasoning is essential not only to guide replanning but also to enable explanation generation.

Plan execution monitoring is commonly studied under the assumption of full observability of the current state. However, privacy concerns, high cost of sensors, unreliability of sensors, or demanding environmental conditions (i.e., tight spaces, high temperatures, or corrosive environments) may render full observability infeasible for certain applications. In such cases, plan execution monitoring needs to be performed under *partial observability*, as only a portion of the current state can be monitored at any given time.

Plan execution monitoring under partial observability is challenging, as all three reasoning tasks of prediction, diagnosis, and replanning need to be significantly extended to accommodate for unobservable states. In particular, for discrepancy detection under partial observability, it is not possible to compare the expected and the observed states since full state observations are not available. Therefore, a discrepancy check is required to be performed based on the monitored fluents. Similarly, during diagnostic reasoning, one should generate explanations due to the discrepancies related to the monitored fluents. Finally, for replanning, the unobservable part of the current state needs to be predicted, taking the diagnosis into account and ensuring consistency with the monitored fluents, such that a new plan can be computed.

Manuscript received 23 August 2021; accepted 12 October 2021. Date of publication 22 November 2021; date of current version 8 August 2022. This paper was recommended for publication by Associate Editor J. Kober and Editor W. Burgard upon evaluation of the reviewers' comments. (*Corresponding author: Volkan Patoglu.*)

The authors are with the Faculty of Engineering and Natural Sciences, Sabanci University, 34956 Istanbul, Turkey (e-mail: gokaycoruhlu@sabanciuniv.edu; esraerdem@sabanciuniv.edu; vpatoglu@sabanciuniv.edu).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TRO.2021.3123840>.

Digital Object Identifier 10.1109/TRO.2021.3123840

In this study, we present a formal method that extends model-based diagnostic reasoning to perform under partial observability and considers multiple observations, such that a possible hypothesis that explains observed discrepancies can be revised as new observations become available. These extensions not only enable the applicability of model-based plan execution monitoring to more realistic scenarios but also improve the autonomy and robust plan execution of robots since it allows for the system to recover from incorrectly diagnosed failures without any external intervention.

We formulate all the reasoning tasks required for plan execution using the expressive logic-based language of Answer Set Programming (ASP)—a knowledge representation and reasoning paradigm based on answer set semantics [3]–[5]—and solve them utilizing efficient ASP solvers. ASP has been used in various applications [6], including robotics [7]. We experimentally evaluate our methods over large sets of problem instances to show the effectiveness of our novel diagnostic reasoning approach and usefulness of plan execution monitoring algorithm.

Contributions: Our contributions can be summarized as follows.

- 1) We present a formal method to *predict* states under partial observability. Our method integrates low-level feasibility checks (i.e., existence of collision-free trajectories) into high-level action descriptions and, thus, can eliminate incorrect predictions of states reached by these actions. It also takes into account previous observations and diagnoses. We use hybrid prediction for three purposes: to detect discrepancies, to check their relevancies, and to infer a full current state for replanning. In the first case, hybrid prediction is utilized to compute the full expected state after executing some part of the plan from the initial state. In the second case, hybrid prediction is utilized to compute the full expected state after executing the rest of the plan from the partially observed current state. In this case, our method infers a full valid state from the current observations, which is closest to the expected state. In the third case, hybrid prediction is utilized to compute the full expected state after executing some part of the plan from the initial state, considering the current observations and diagnosis.
- 2) We propose a formal method for *diagnostic reasoning* under partial observability to generate meaningful explanations for the relevant discrepancies. Our method not only integrates low-level feasibility checks into high-level diagnostic reasoning but also enables revisions of previous diagnoses such that more relevant explanations can be generated as further observations become available. We use hybrid diagnostic reasoning to identify the minimum number of robotic components that are most likely to cause the detected relevant discrepancy when they are broken. Our method also provides further explanations as to which actions of the plan fail and when, due to the inferred diagnosis.
- 3) We present a formal method for *replanning* under partial observability that generates new plans starting from the current state (with predicted unobservable part) and

leading to the goal state, guided by the diagnosis with explanations. Our replanning method also integrates low-level feasibility checks into high-level task planning to ensure that the computed plans are executable in the real world. Our replanning method also considers repairing a minimum number of broken parts, if needed.

- 4) Based on these ASP-based formal methods to detect discrepancies, generate diagnoses with explanations, and compute (re)plans, we introduce a novel *plan execution monitoring algorithm* that can be used under partial observability. Our algorithm can be used autonomously as well as interactively to involve the user in decisions and get their feedback (e.g., which diagnosis is more likely or which repairs are more preferable during replanning).
- 5) We experimentally evaluate our plan execution monitoring algorithm over large sets of problem instances in a service robotics domain that involves multiple mobile manipulators to investigate its scalability and effectiveness with respect to different variations of diagnostic reasoning (i.e., with or without considering former observations and diagnoses) and replanning (i.e., with or without guidance by diagnosis).

Outline: The rest of the article is organized as follows. Section II provides a comparative review of the relevant studies on plan execution monitoring. Section III overviews the proposed plan execution monitoring approach. Section IV reviews ASP and presents a formal definition of the planning problem in ASP. Section V introduces a formal definition of the diagnosis problem using ASP and presents novel formal methods to compute a most-probable minimum-cardinality diagnosis and to generate explanations for this diagnosis. Section VI introduces a formal definition of the prediction problem using ASP and presents formal methods to detect a discrepancy and to determine its relevancy. Section VII introduces a formal definition of guided replanning and presents formal methods to compute replans guided by the most recent diagnosis and possibly by repairing a minimum set of robot components. Section VIII presents the pseudo-code and details of the plan execution monitoring algorithm. Section IX presents a service robotics case study, where the domain description is explained in detail and illustrative examples demonstrating each module of our plan execution monitoring algorithm are introduced. Section X presents evaluations of our methods over large sets of problem instances to show the effectiveness of our novel diagnostic reasoning approach and the usefulness of the plan execution monitoring algorithm. Finally, Section XI concludes the article.

II. RELATED WORK

We are concerned with execution monitoring that not only detects discrepancies/failures but also finds out the reasons/diagnosis of these discrepancies/failures for a better-guided replanning. Therefore, we examine the related work in two parts: studies about execution monitoring that utilize some useful information for replanning and studies about diagnostic reasoning.

Before the detailed discussions, let us clarify some terminology. We consider dynamic domains where the world states change over time due to direct or indirect effects of actions of the robots.

We understand discrepancies in the spirit of [8] as differences between an observed state and an expected state of the world (e.g., the table is empty while a glass was expected to be on it); discrepancies may lead to a failure of a plan. In that sense, our understanding of a discrepancy is different from “faults” [9], which are abnormal conditions or defects of physical components (e.g., deformation of a robot wheel); faults of a component may lead to the failure of the functionality of that component.

We consider a special type of faults, “broken components of robots,” as the cause of a discrepancy that leads to a plan failure. In that sense, our understanding of diagnostic reasoning is similar to fault diagnosis. Different from many various existing approaches for fault diagnosis [9], we utilize model-based methods of automated reasoning.

As for plan failures, we consider failures that are caused by broken components. Other types of failures, such as failures due to human intervention [10], failures due to collisions with movable obstacles whose presence and location are not known in advance [11], and failures due to heavy objects that cannot be lifted alone [11], are investigated in complementary studies. Also, we do not consider action failures due to unreliable/incomplete execution of actions, which are usually handled at the low-level utilizing behavior trees [12] or finite state machines via temporal logic [13], [14]. Finally, we do not present the low-level control aspects and physical implementation details to avoid distracting readers from the main contributions of this study.

Execution monitoring with more informative failures/discrepancies. There is related work (starting with SHAKEY [15]) on execution monitoring that utilizes replanning upon detection of action failures; for a comprehensive summary of these approaches, we refer the reader to the thesis by Fritz [16] from the perspective of AI and to [17] from the perspective of robotics and control.

Some execution monitoring approaches understand a failure as the failure of the current action’s execution in a plan, identify the reasons for the failure in terms of the unsupported preconditions and/or effects, and then utilize this knowledge while replanning [18]–[20]. For instance, Ambros-Ingerson and Steel [18] use a production system architecture where IF–THEN rules are used to map “flaws” (e.g., unsupported preconditions of a failed action) to “fixes” while replanning. Fichtner *et al.* [19] specify preconditions and effects of actions under the assumption that some properties are not “abnormal” and thus identify reasons of an action failure in terms of these abnormalities and utilize this information during replanning. Similarly, Winkler *et al.* [20] describe possible error cases and how to react to them by high-level macros in goal definitions.

Some execution monitoring approaches consider discrepancies between the expected state and the observed state [8], [21], [22]. Discrepancies involve action failures, and more information; therefore, they can be useful in various ways depending on how they are defined and utilized. For instance, in addition to action failures, Lemai and Ingrand [21] consider temporal/resource

conflicts (e.g., when an action takes longer time than planned) and perform plan repairs (e.g., postponing the remaining actions). De Giacomo *et al.* [8] understand a discrepancy as a failure of the remaining part of the plan and perform replanning only when a discrepancy is relevant. In another work [23], where the goal is to compute a policy “in the now,” the authors do not consider discrepancies but “reconsiderations” (e.g., when the precondition of an action is not achievable) to replan. In another study [24], the authors compute all the expected states during the execution of a plan in advance by a stochastic prediction function and utilize them to detect discrepancies between the observed ones during plan execution; if a relevant discrepancy is detected, then the prediction function is updated.

Some execution monitoring approaches further try to find explanations for discrepancies [22], [25], [26]. Zhang *et al.* [22] define failures (e.g., states where a cup is not observed to be in the main container) and utilize plan recovery methods using defaults (e.g., if the cup is not in the main container, then, by default, it is in the auxiliary container). In another study [25], due to lack of support for defaults, the notion of “assumptive actions” is introduced to allow the system to generate assumptions about the initial state that might cause the detected discrepancy between the observed state and the expected state. Eiter *et al.* [26] define a diagnosis of discrepancy as a “point of failure,” where execution of a sequence of actions evolves differently than expected. This additional knowledge explains the detected discrepancy and allows the agent to reverse its plan until the point of failure and re-execute or repair the rest of the plan.

In our execution monitoring framework, similar to [8], [21], [22], [25], and [26], we consider discrepancies between the expected state and the observed state but with respect to a set of monitored fluents (under partial observability); so the observed state may be incomplete. Also, instead of continuous monitoring, we propose a general approach to check for discrepancies intermittently; so the failure of a condition may not be observed at all until a discrepancy is detected.

Similar to [8], we check for the relevancy of a discrepancy for the execution of the rest of the plan and perform replanning when the discrepancy is relevant. Different from the studies above, we identify possible causes of discrepancies by means of sets of broken components of robots and points of failures. We perform a novel method of diagnostic reasoning to find such possible causes under partial observability. Furthermore, we generate explanations based on such diagnoses about which actions could not be executed successfully due to these diagnoses. The additional information due to the computed diagnoses and explanations is utilized to guide replanning.

Diagnosis of discrepancies. Diagnoses of discrepancies are defined in various ways in the literature. For instance, according to Reiter and de Kleer [27], [28], a diagnosis of a discrepancy is described by a set of system components that are broken. As mentioned above, Eiter *et al.* [26] define a diagnosis of discrepancy as a point of failure. Roos and Witteveen [29] define a diagnosis of a discrepancy as a combination of some actions executed so far such that if these actions are qualified as abnormal, the observed plan state is compatible with the predicted plan state. In this study, we define diagnoses of discrepancies

by means of sets of broken components of robots and points of failures. Different from the studies [27], [28], we consider dynamic domains where the world states change over time, instead of a static system.

Concerning dynamic domains, diagnosis of failures or discrepancies by means of broken components is studied in the literature [30]–[32]. Let us examine these studies in detail.

McIlraith *et al.* [30] consider hybrid systems modeled using automata and temporal causal graphs; their goal is to identify a most-probable diagnosis for a detected discrepancy. Different from this study, we do not model a robot as a hybrid system; instead, we model the robotic domain as a dynamic environment, where continuous variables regarding the robot are not represented explicitly but embedded in the high-level description of the domain. Our goal in execution monitoring is to find a most-probable diagnosis among the ones with minimum cardinality.

Balduccini and Gelfond [31] and Baral *et al.* [32] model dynamic domains using nonmonotonic formalisms, A-Prolog (based on answer set semantics [3], [4]) and Action Language $\mathcal{L}$ [33]. They introduce exogenous actions (e.g., a “break” action that causes parts to malfunction) in the domain descriptions to identify broken components that might cause discrepancies. Our approach is similar in that we use the nonmonotonic formalism of ASP [34]–[36], also called A-Prolog, to model a dynamic domain. Different from these studies, our approach does not need to introduce such a dummy action since we can represent changes that do not involve a robotic action. Furthermore, unlike these approaches, our domain description is hybrid where the feasibility checks are embedded.

For replanning purposes, Balduccini and Gelfond [31] and Baral *et al.* [32] introduce exogenous “repair” actions in the domain descriptions to allow repairs of broken components. Different from these studies, our approach does not need to introduce such a dummy action since we can allow repairs using weak constraints. To deal with replanning from a partially observable state, Baral *et al.* [32] consider sensing actions (i.e., to gather the missing knowledge about all the unobserved fluents via sensors) and conditional planning. In our study, we do not consider sensing actions as the world is assumed to be partially observable even with the sensors and, thus, present a method based on sequential planning, although our framework can be extended to conditional planning (if the world can be observed via additional sensors), as discussed in our earlier studies [37]–[39].

In this study, we are also concerned with generating explanations for relevant discrepancies that provide information not only about which action in the plan being executed has failed and when but also what might have caused such a failure in terms of broken parts. As the cause of a discrepancy may have occurred early in the plan execution and may not have been easily detected by sensors, we utilize diagnostic reasoning for explanation generation. In that sense, our approach is more general than the recent studies [40] in robotics that identify the cause of a robotic failure with respect to a taxonomy of failures in terms of the current failed action, previous successful action, and current captured environmental context only. Note that we

do not study explainability of plans [41], [42] but explanation generation for relevant discrepancies in plan execution.

Our earlier studies. The study presented in this manuscript builds on and significantly extends our previous work in [43] in the following ways: 1) We assume that, by default, components are not broken, which allows components being broken at the very initial state, 2) we enable the diagnostic reasoning method to revise its previous explanations whenever new observations become available, and 3) most importantly, we significantly extend all the formal reasoning tasks in [43] to function under partial observability.

A summary of the most distinctive features of our method in comparison with the related work. Our approach:

- 1) works under partial observability and, furthermore, allows the monitored fluents to be partially observed;
- 2) is integrated with low-level feasibility checks;
- 3) generates meaningful hypotheses (i.e., the most-probable ones among the minimum cardinality diagnoses) about possibly broken robotic components;
- 4) generates explanations about which action has failed and when due to these broken parts;
- 5) determines which broken parts need to be repaired such that the goal can be reached;
- 6) revises its previous hypotheses based on new observations;
- 7) does not require a dummy “break” action or a dummy “repair” action.

It is important to emphasize that our contributions emerge from logic-based knowledge representation and automated reasoning and thus utilize expressive logical formalisms and efficient automated reasoners to develop a general formal framework that not only can solve a variety of reasoning problems, including prediction, planning, guided replanning, and diagnostic reasoning but also can orchestrate these reasoning modules by providing feedback in terms of constraints and preferences. For instance, thanks to the expressive formalism of ASP, we can utilize nonmonotonic constructs to express “defaults” (e.g., that every robotic part is assumed to be not broken by default, but it may get broken at any time). We can modularly express state/transition constraints, preferences, multiobjective optimizations, and ramifications, without having to change the existing action descriptions. Here, elaboration tolerance [44] of the representation aids the synergy between different reasoning modules. Furthermore, we can integrate feasibility checks (and motion planning) into planning and other reasoning tasks by utilizing “external atoms” [45], which are more general than “semantic attachments” [46] used for classical planning, as the latter do not allow predicate extensions but only terms as arguments to attachments. This feature of external atoms is, in particular, useful for domains that include multiple movable obstacles.

Therefore, special languages for planning (e.g., PDDL) and the classical planners that support “semantic attachments” can be used for hybrid planning and guided replanning in our framework but only in some cases, and necessitating substantial “surgeries” on operator definitions (e.g., ramifications and concurrency may require introduction of new operators [47], and adding a global constraint may require updating the

preconditions and effects of every operator). However, even under these restrictions, the use of classical planners for diagnostic reasoning is not possible as our approach to diagnostic reasoning heavily relies on defaults.

III. OVERALL IDEA OF OUR PLAN EXECUTION MONITORING FRAMEWORK

Our plan execution monitoring framework is based on formally representing a dynamic robotic domain by means of logical formulas using the nonmonotonic formalism of ASP [48] embedded with external feasibility checks. Utilizing the reasoning mechanisms of ASP, it is capable of a rich set of hybrid reasoning tasks. Let us briefly give the overall idea of our framework by underlining these capabilities.

Given an initial state and goal conditions, the planning agent starts with *computing a hybrid plan of actions* taking into account the logical descriptions as well as the feasibility checks of actions. Robots start executing the actions in this plan and occasionally perform (partial) observations of the environment.

Whenever a new observation is received, the monitoring agent verifies that the plan has evolved as expected by first *predicting the expected state* and then *checking the consistency of the partially observed current state against the expected one*.

If a discrepancy is detected, then the monitoring agent checks the relevancy of this discrepancy to the rest of the plan by first *inferring a valid full state description for the partially observed current state* and then *checking the validity of the rest of the plan from the current observed state*. If no discrepancy is detected or the detected discrepancy is not relevant, the execution of the plan continues.

If the detected discrepancy is relevant, then the diagnosis agent identifies possible causes of this discrepancy by *diagnosing broken parts of the robots* and *generating further explanations for the discrepancy in terms of failed actions*. For an observed discrepancy, there can be more than one diagnosis or explanation obtained by such *hypothetical reasoning*; so the diagnosis agent finds a min-cardinality diagnosis with more likelihood of failure and an explanation for it.

Once such a diagnosis and explanation are computed, the replanning agent *finds a new hybrid plan from the current state (inferred from the partially observed state) to the goal, guided by the constraints (obtained from inferred diagnoses)*. If there exists no such plan, the replanning agent *infers a repair by identifying the minimum number of broken parts that can be repaired such that the goal can be achieved by guided replanning*.

Our plan execution monitoring framework is autonomous; yet, it can be used in an interactive mode as well. When the diagnosing agent computes a set of most-probable min-cardinality diagnoses with explanations, the monitoring agent may consult an expert to pick one of them.

All of the reasoning tasks are performed over a hybrid formulation of the robotic domain that combines high-level logical reasoning with low-level feasibility checks, possibly based on probabilistic methods.

We provide a detailed description of these hybrid reasoning tasks in the following sections. We describe the plan execution monitoring algorithm in detail in Section VIII.

IV. PLANNING OF ACTIONS

Dynamic domains can be modeled as transition systems—directed graphs where nodes denote the world states of the domain and edges denote the transitions between these states caused by occurrences or nonoccurrences of actions in that domain.

We can represent the states and transitions denoted by a transition system in ASP [48] by means of formulas, called *rules*, of the form $Head \leftarrow Body$, where *Head* and *Body* are propositional formulas where each literal (i.e., a propositional atom q or its negation $\neg q$) is possibly preceded by default negation *not*. If *Head* is $\perp$, then the rule is called a *constraint*. A finite set of rules is called a *program*. The models of a program are called *answer sets*, and they can be computed by special solvers, like CLINGO [49].

The idea is to solve planning, prediction, and diagnosis problems over a transition system using the formal framework of ASP: Once the transition system modeling a dynamic domain is represented by a program, we combine this program with further constraints to solve a reasoning problem (e.g., planning). For instance, for a planning problem, the constraints describe the initial and goal states. Then the answer sets computed for the combined program characterize solutions (e.g., plans for the planning problems).

Before we formally define the planning and diagnosis problems in ASP, let us summarize how a transition system can be represented in ASP.

A. Representing Transition Systems in ASP

A planning domain description $\mathcal{D}^*$ of a dynamic domain as a transition system is formulated in ASP over a set $\mathcal{F}$ of fluent constants and a set $\mathcal{A}$ of action constants, with respect to a given upper bound k on time steps. This description relies on different forms of rules [7, Sec. 2]. Let us briefly go over some important types of rules relevant to our study.

For a formula H and an index i (for time step), let us denote by $H(i)$ the formula obtained from H by replacing every atom q by $q(i)$. Here, $H(i)$ expresses that the formula H holds at time step $0 \leq i \leq k$.

Effect rules: Direct outcomes of actions are expressed with *effect rules* of the form

$$E(i+1) \leftarrow A(i), F(i) \quad (1)$$

where A is a conjunction of action atoms, E is a fluent literal, and F is a conjunction of fluent literals. This rule indicates that if the actions in A are executed at time step i where F holds, then at the next state, E holds. For instance, the following effect rule describes an effect of a “move” action of a mobile robot r navigating to a location l at time step i

$$at(r, l, i+1) \leftarrow move(r, l, i).$$

It expresses that, as a direct effect of this action, the location of robot r changes to l at the next time step $i + 1$.

Precondition rules: Preconditions of actions are expressed with *precondition rules* of the form

$$\leftarrow A(i), F(i), \text{not } G(i) \quad (2)$$

where A is a conjunction of action atoms, and F and G are conjunctions of fluent literals. The precondition rule above expresses that to execute an action A at time step i at a state where F holds, the action's preconditions G must hold. For instance, according to the following precondition rule:

$$\leftarrow \text{move}(r, l, i), \text{at}(r, l, i)$$

action $\text{move}(r, l)$ is possible if the robot is not already at the destination location l .

Hybrid rules: A *hybrid rule* is a rule where the right-hand side of $\leftarrow$ includes external atoms. External atoms [45] are not fluent or action constants; their truth values are computed externally (out of ASP).

These rules are important for robotics applications since low-level feasibility checks for each action can be computed externally and then integrated into a transition system description by means of external atoms. For instance, the following hybrid precondition rule ensures that, at time step i , a robot r can move from its current location x to its destination location l if there is a collision-free trajectory between them

$$\leftarrow \text{at}(r, x, i), \text{move}(r, l, i), \text{not } \& \text{moveIsFeasible}[r, l, x]().$$

The external atom $\& \text{moveIsFeasible}[r, l, x]()$ passes r, l, x as inputs to the external computation (e.g., a Python program) that calls a motion planner to check the existence of a collision-free trajectory for r from x to l and then returns the result of the computation to the precondition rule.

Weak constraints: The ASP programs can be augmented with “weak constraints” [50]—expressions of the following form:

$$:\sim \text{Body}(t_1, \dots, t_n)[w@p, t_1, \dots, t_n].$$

Here, $\text{Body}(t_1, \dots, t_n)$ is a formula with the terms $t_1, \dots, t_n$. Whenever an answer set for an ASP program satisfies $\text{Body}(t_1, \dots, t_n)$, the tuple $\langle t_1, \dots, t_n \rangle$ contributes a cost of w to the total cost function of priority p . The ASP solver tries to find an answer set with the minimum total cost. For instance, the following weak constraint

$$:\sim \text{move}(r, l, i), \text{broken}(r, \text{base}, i)[1@2]$$

instructs CLINGO to compute an answer set that does not include both $\text{move}(r, l, i)$ and $\text{broken}(r, \text{base}, i)$, if possible, with the assumption that robot base is broken and, therefore, cannot perform move action properly. However, if CLINGO cannot find such an answer set, it is allowed to compute an answer set with these atoms $\text{move}(r, l, i)$ and $\text{broken}(r, \text{base}, i)$ but with an additional cost of 1. Weak constraints are considered by CLINGO according to their priorities.

Nondeterministic choices: Transition systems rely on the possibility of *nondeterministic choices* about the occurrences of actions and initial values of fluents. For instance, the following

choice rule

$$\{\text{move}(r, l, i)\} \leftarrow$$

expresses that a robot r can choose to move to a location l at time step i . The possibility of this choice is subject to the action's preconditions. Similarly, the following choice rule expresses that initial locations l of objects o can be arbitrary at time 0

$$\{\text{at}(o, l, 0)\} \leftarrow .$$

State constraints: The nodes of a transition system characterize valid states described with *state constraints*. For instance, the following state constraint ensures the existence of a location for every object o at every time i

$$\leftarrow \{\text{at}(o, l, i) : \text{loc}(l)\} 0$$

whereas the following state constraint ensures the uniqueness of a location for every object o :

$$\leftarrow 2\{\text{at}(o, l, i) : \text{loc}(l)\}.$$

Here, the numbers 0 and 2 denote the upper and the lower bounds on the cardinality of subsets of $\{\text{at}(o, l, i) : \text{loc}(l)\}$.

For detailed examples of representing robotic domains in ASP, we refer the reader to our earlier studies [11], [51], [52].

B. Defining Planning Problems in ASP

A planning problem in a dynamic domain, characterized by an initial state s_0 , a goal state s_g , and a maximum length k of a plan, asks for a path P in the transition system modeling that domain such that the path starts at a node that denotes s_0 , ends at a node that denotes s_g , and the length of the path is less than k . The following provides a formal definition of a plan when the transition system of a dynamic domain is formalized in ASP by a set $\mathcal{D}^k$ of rules as described above, over a set $\mathcal{F}$ of fluent constants and a set $\mathcal{A}$ of action constants, where the time steps range over $0 \dots k$.

Let us characterize a state $s \subseteq \mathcal{F}$ by a conjunction F_s of fluent atoms that hold at that state and the negations of fluent atoms that do not hold at that state. For every action $A \subseteq \mathcal{A}$, let us denote by E_A the conjunction of action atoms in A and negations of action atoms in $\mathcal{A} - A$; E_A will be used to express that only the elementary actions in A are executed.

Definition 1: A planning problem $\mathcal{P}$ is characterized by a tuple $\langle \mathcal{D}^k, s_0, s_g, k \rangle$. A solution of $\mathcal{P}$ (called a *plan*) is a sequence $\langle A_0, \dots, A_{n-1} \rangle$ ($n < k$) of actions such that $\mathcal{D}^k$ when combined with the following rules (called a *planning query*) has an answer set:

$$\begin{aligned} &\leftarrow \text{not } F_{s_0}(0) \\ &\leftarrow \text{not } F_{s_g}(k) \\ &\leftarrow \text{not } E_{A_i}(i) \quad (0 \leq i \leq n-1). \end{aligned} \quad (3)$$

The first and second constraints in (3) ensure that the plan starts at the given initial state and reaches the given goal state. The third constraint ensures that only the actions of the plan are executed.

Based on the definition above, plans of actions can be computed using the answer set solver CLINGO.

V. DIAGNOSTIC REASONING AND EXPLANATIONS

During the plan execution, various discrepancies may occur between the expected state and the observed state and these discrepancies may result in plan failures. In the absence of knowledge of what went wrong, the execution of the newly computed plan is also likely to fail in some cases and the overall plan executed to reach the goal is expected to be longer. Diagnostic reasoning is required to identify and recover from such failures as early as possible.

In this study, we assume that discrepancies may only occur due to malfunctioning of robot components. Then, a discrepancy can be diagnosed by a set of broken robot components.

To be able to perform diagnostic reasoning, the action domain description $\mathcal{D}^k$ used for planning has to be updated since it formalizes states and transitions under the assumption that no anomalies occur.

A. Modifying the Action Domain Description Into a Diagnosis Domain Description

An action domain description $\mathcal{D}^k$ does not describe what will happen if a component of a robot is broken: if a robot part is broken, then some precondition of an action in the plan may not hold anymore, and, thus, the plan simply fails and no further knowledge becomes available for reasoning. Therefore, for the purpose of diagnosing the reasons of such failures, we systematically transform the planning domain description $\mathcal{D}^k$ into a diagnosis domain description. Our method is based on the transformation suggested by [43] with some modifications.

Let M be a set of fluent constants ($M \subseteq \mathcal{F}$) that are monitored. Let $\mathcal{R}$ denote the set of pairs of all robots and their components that may get broken. For instance, $\mathcal{R} = \{(robot1, base), (robot2, arm)\}$ describes that the base of Robot 1 and the arm of Robot 2 may be broken. Let *disables* be a relation $2^{\mathcal{R}} \times \mathcal{A} \times \mathcal{F}$ that describes which actions are affected when a set of robotic parts are broken. For instance, *disables*(X, a, F) represents that the effect of action a on F is disabled, if X are broken.

Suppose that a relevant discrepancy is detected at time step $t \leq k$ and we want to find a diagnosis for it. We transform $\mathcal{D}^k$ to a new description $\mathcal{D}_{diag}^k$ as follows, subject to the constraint that the time steps range over $0 \dots t$.

Stage 1: A fluent constant of the form *broken*(r, p) is introduced for each pair $(r, p) \in \mathcal{R}$, and the following rules are added to $\mathcal{D}^k$:

$$\begin{aligned} \neg broken(r, p, i) &\leftarrow not\ broken(r, p, i) \\ broken(r, p, i+1) &\leftarrow not\ \neg broken(r, p, i+1), \neg broken(r, p, i) \\ broken(r, p, i+1) &\leftarrow broken(r, p, i) \\ \{broken(r, p, 0)\}. \end{aligned}$$

These rules express that every robotic part (r, p) is assumed to be not broken by default but may get broken at any time step i . If a robotic part gets broken, it remains broken. Moreover, different from [43], a robotic part may be broken initially.

Stage 2: A fluent constant *pre*(A) is introduced for each action $A \subseteq \mathcal{A}$ to identify preconditions of A . By default, the value of *pre*(A) is true for every action A ; so the following rule is added

to $\mathcal{D}^k$:

$$pre(A, i) \leftarrow not\ \neg pre(A, i).$$

Every precondition rule (2) of A in $\mathcal{D}^k$ is replaced by

$$\neg pre(A, i) \leftarrow F(i), not\ G(i).$$

Every effect rule of A (1) in $\mathcal{D}^k$ is replaced by

$$E(i+1) \leftarrow A(i), F(i), pre(A, i).$$

These rules express that, if the preconditions of an action A holds, then its effects are observed as expected.

Stage 3: Let A_R denote the robots that take place in the execution of action $A \in \mathcal{A}$ which may get broken (i.e., for every $r \in A_R$, there is a pair (r, p) in $\mathcal{R}$). The following rule is introduced to $\mathcal{D}_{diag}^k$ for each effect rule (1) in $\mathcal{D}^k$ such that *disables*(A_R, A, F) holds:

$$E(i+1) \leftarrow A(i), F(i), \bigwedge_{r \in A_R, (r, p) \in \mathcal{R}} \neg broken(r, p, i).$$

These rules express that the direct effects of actions are observed only if the relevant robotic parts are not broken.

B. Min-Cardinality Diagnosis

Once a relevant discrepancy is detected at time step t after executing the part $P_{<t} = \langle A_0, \dots, A_{t-1} \rangle$ of the plan at the initial state s_0 , diagnostic reasoning can be performed as follows:

Definition 2: A diagnosis problem, DP , is characterized by a tuple $\langle \mathcal{D}_{diag}^k, \mathcal{R}, s_0, P_{<t}, O_t^M, X_{<t} \rangle$ where we have the following.

- 1) O_t^M is a set of earlier and current observed states o_i^M of monitored fluents in M at time step $i \leq t$.
- 2) $X_{<t}$ is a set of triples (r, p, i) ($i < t - 1$) that describe the parts p of robots r ($(r, p) \in \mathcal{R}$) that were earlier diagnosed as broken.

A solution of DP is a set X_t of triples (r, p, j) that describe robotic components $(r, p) \in \mathcal{R}$ that are assumed to be broken at time step $j < t$ such that $\mathcal{D}_{diag}^k$ when combined with the following rules (called a *diagnostic query*) has an answer set

$$\begin{aligned} &\leftarrow not\ F_{s_0}(0) \\ &\leftarrow not\ E_{A_i}(i) \quad (0 \leq i \leq t-1) \\ &\leftarrow not\ F_{o_i^M}(i) \quad (o_i^M \in O_t^M) \\ &\leftarrow not\ broken(r, p, i) \quad ((r, p, i) \in (X_t \cup X_{<t})) \\ &\leftarrow not\ \neg broken(r, p, l) \\ &\quad ((r, p) \in \mathcal{R}, l < t, (r, p, l) \notin (X_t \cup X_{<t})). \end{aligned} \tag{4}$$

If DP has a solution X_t , then we say that $X_t \cup X_{<t}$ is a *diagnosis* for the relevant discrepancy detected at time step t after executing $P_{<t}$.

The diagnostic query (4) checks if every observed state in O_t^M can be reached from the initial state s_0 by executing the plan $P_{<t}$ if the parts of the robots in X_t and $X_{<t}$ were broken at specified times. If the program $\mathcal{D}_{diag}^k$ combined with rules (4) has an answer set, then $X_t \cup X_{<t}$ is a potential cause for the detected discrepancy.

Definition 2 can be used to implement three novel approaches to diagnostic reasoning.

- 1) *Augmented Diagnosis*: A diagnosis is generated about the broken parts based on the current observation, while imposing the validity of all of the previous hypotheses. This approach can be implemented by the following settings: $X_{<t}$ is set to the previously diagnosed broken parts before time step t and O_t^M contains only the last observation o_t^M .
- 2) *Reset Diagnosis*: A diagnosis is generated about the broken parts based on the current observation, without considering any of the previous hypotheses. This approach can be implemented by the following settings: $X_{<t} = \emptyset$ and O_t^M contains only the last observation o_t^M (i.e., $O_t^M = \{o_t^M\}$). The reset diagnosis approach is a generalization of the approach presented in [43] to function under partial observability.
- 3) *Revised Diagnosis*: A diagnosis is generated about the broken parts based on the current observation and the previous observations, while updating the previous hypothesis as necessary. This approach can be implemented by the following settings: $X_{<t} = \emptyset$ and O_t^M contains the last observation o_t^M as well as the previous observations o_i^M ($i < t$).

In general, more than one diagnoses can be generated for a discrepancy. In practice, focusing on diagnoses that include the minimum number of broken parts may be more reasonable since malfunctioning of multiple robot parts at the same time is unlikely. We can find a min-cardinality diagnosis by adding the following weak constraints to the diagnostic query (4):

$$:\sim broken(r, p, i) \quad (i < t, (r, p) \in R).[1@2, r, p]. \quad (5)$$

The rules (4) $\cup$ (5) define a *min-cardinality diagnostic query*.

C. Most-Probable Min-Cardinality Diagnosis

For a plan utilizing several robots, where each of them has several components, there may be more than one (min-cardinality) diagnosis for a detected relevant discrepancy. In such cases, our execution monitoring algorithm can utilize *a priori* knowledge about the likelihood of each robotic component getting broken to identify the most-probable diagnosis out of all (min-cardinality) diagnoses.

We express such *a priori* knowledge by atoms of the form $prob_b(r, p, w)$ where w is a positive integer that describes the likelihood of a component p of a robot r getting broken. Then, we add the following weak constraints to the min-cardinality diagnostic query (4) $\cup$ (5) to find a most-probable min-cardinality diagnosis:

$$:\sim broken(r, p, i), prob_b(r, p, w)(i < t, (r, p) \in R).[-w@1, r, p].$$

Note that the priority of the weak constraint above is less than that of (5) to ensure that a most-probable diagnosis is found among the min-cardinality diagnoses.

D. Explanation Generation

While diagnoses explain the reasons of relevant discrepancies in terms of broken robotic components, further explanations can be generated to include the actions whose effects have not been

observed as expected due to these diagnoses. For that purpose, we can extend the diagnostic reasoning problem as follows.

For every action constant $A \in \mathcal{A}$, let $A_R \subseteq \mathcal{R}$ be the set of all pairs (r, p) of robots r and their parts p such that r takes part in executing the action A and that $disables(A_R, A, F)$ holds for some fluent constant $F \in \mathcal{F}$.

Definition 3: Let $DP = \langle \mathcal{D}_{diag}^t, \mathcal{R}, s_0, P_{<t}, O_t^M, X_{<t} \rangle$ be a diagnosis problem. A *diagnosis with an explanation* for DP is a set X^E of tuples (r, p, A, i) where $(r, p) \in A_R$ and $i < t$, such that $\mathcal{D}_{diag}^t$ when combined with the diagnostic query (4) where $X_t \cup X_{<t} = \{(r, p, i) | (r, p, A, i) \in X^E\}$, and the following rules has an answer set:

$$\begin{aligned} explains(r, p, A, i) &\leftarrow A(i), broken(r, p, i) \\ ((r, p, A, i) &\in X^E, i < t). \end{aligned} \quad (6)$$

The atoms of the form $explains(r, p, A, i)$ that appear in an answer set for $\mathcal{D}_{diag}^t \cup (4) \cup (6)$ express which actions have failed because of which broken components.

VI. DISCREPANCIES AND RELEVANCY

Diagnostic reasoning is applied if a discrepancy is detected and found relevant for the rest of the plan. Let us define these concepts more formally.

A. Predicting the Expected State

A discrepancy (if there exists one) can be detected by comparing the expected state and the observed state after the execution of some part of a plan. We can predict an expected state by simulation of the part of the plan considering “believed-to-be-broken” robot components, while an observed state can be obtained by monitoring fluents with the help of sensors during the execution of the plan.

Definition 4: A prediction problem PP is characterized by a tuple $\langle \mathcal{D}_{diag}^t, s_0, P_{<t}, O_t^M, X_{<t} \rangle$. A solution of PP (called an *expected state*) is a state s_t such that $\mathcal{D}_{diag}^t$ when combined with the following rules has an answer set:

$$\begin{aligned} &\leftarrow not F_{s_0}(0) \\ &\leftarrow not F_{s_t}(t) \\ &\leftarrow not E_{A_i}(i) \quad (0 \leq i \leq t-1) \\ &\leftarrow not F_{o_i^M}(i) \quad (o_i^M \in O_t^M) \\ &\leftarrow not broken(r, p, i) \quad ((r, p, i) \in X_{<t}) \\ &\leftarrow not \neg broken(r, p, i) \\ &\quad ((r, p) \in \mathcal{R}, i < t-1, (r, p, i) \notin X_{<t}). \end{aligned} \quad (7)$$

The first and second constraints of (7) ensure that the plan starts at the given initial state and reaches the expected state. The third constraint ensures that only the actions of the plan are executed. The fourth constraint takes into account the earlier and current observations. The fifth and sixth constraints indicate that only the (r, p) pairs in $X_{<t}$ are assumed to be broken and the rest of the (r, p) pairs are functioning properly.

B. Discrepancies

A discrepancy (if there exists one) is detected by comparing the expected state with the current observations.

Let $\mathcal{D}_{state}$ be the union of the state constraints and the nondeterministic choice rules for initial values of fluents in $\mathcal{D}^0$ for time step 0. The answer sets for $\mathcal{D}_{state}$ characterize the states of the transition diagram defined by $\mathcal{D}^k$.

Definition 5: Let s_e be the expected state, and o^M be a set of observations of monitored fluents in M . There is a *discrepancy* between s_e and o^M if the program $\mathcal{D}_{state}$ when combined with the following rules does not have an answer set:

$$\begin{aligned} &\leftarrow \text{not } F_{s_e}(0) \\ &\leftarrow \text{not } F_{o^M}(0). \end{aligned}$$

C. Inferring the Current State From Partial Observations

To check the relevancy of a discrepancy for the rest of the plan execution and then to be able to replan if needed, we need to infer a valid full state from the partially observed current state, while respecting the state constraints of the robotic domain. Meanwhile, among all possible current states, we prefer the ones that are closest to the expected state.

For every state s , let us denote by $l(s)$ the set of fluent literals that hold at state s .

Definition 6: A valid state generation problem VSG is characterized by a tuple $\langle \mathcal{D}_{state}, s_e, o^M \rangle$ where we have the following.

- 1) $\mathcal{D}_{state}$ is an ASP description of valid states of a robotic domain.
- 2) s_e is an expected state.
- 3) o^M is an observed state of monitored fluents in M .

A solution of VSG is a *valid full state* s_v with respect to s_e and o^M such that $\mathcal{D}_{state}$ when combined with the following rules has an answer set

$$\begin{aligned} &\leftarrow \text{not } F_{s_v}(0) \\ &\leftarrow \text{not } F_{o^M}(0) \\ &:\sim l_e(0). \quad [-1@1] \quad (l_e \in l(s_e)). \end{aligned} \quad (8)$$

The first two constraints of (8) ensure that the state s_v is an inferred valid state where the current observations o^M hold. The weak constraints maximize the number of common fluent literals between the expected state s_e and the inferred valid state s_v by minimizing their differences.

D. Determining the Relevancy of a Discrepancy

Sometimes observed discrepancies may be irrelevant to the rest of the plan, i.e., they may not prevent the rest of the plan from reaching a goal state. Therefore, when a discrepancy is detected, the monitoring agent takes the relevancy of a discrepancy into consideration before performing any diagnosis or replanning tasks.

Definition 7: A discrepancy relevancy check problem RC is a decision problem characterized by a tuple $\langle \mathcal{D}_{diag}, s_t, o_t^M, s_v, P_{\geq t}, s_g, X_{<t} \rangle$ where we have the following.

- 1) s_t is the expected current state at time step t (i.e., a solution for the prediction problem $\langle \mathcal{D}_{diag}, s_0, P_{<t}, O_t^M, X_{<t} \rangle$).
- 2) o_t^M is a set of current observations of monitored fluents in M at time step t such that there is a discrepancy between s_t and o_t^M .
- 3) s_v is a valid full state with respect to s_t and o_t^M .

- 4) $P_{\geq t} = \langle A_t, \dots, A_{n-1} \rangle$ is the rest of the plan to be executed.

The discrepancy relevancy check problem RC returns True (i.e., the discrepancy between s_t and o_t^M is relevant for the execution of $P_{\geq t}$ at s_v , subject to $X_{<t}$) if the goal state s_g is not the expected state for the prediction problem $\langle \mathcal{D}_{diag}, s_v, P_{\geq t}, \{ \}, \{ (r, p, 0) | (r, p, i) \in X_{<t} \} \rangle$. Otherwise, RC returns False.

VII. GUIDED REPLANNING AND REPAIRS

After a relevant discrepancy is detected (Definition 7) at time step $t \leq n$ and a possible diagnosis X is generated for it (Definition 2), we can perform replanning to reach the goal respecting the upper bound k on the total plan length. Replanning can be guided by the generated diagnosis leading to more meaningful plans by ensuring that robots do not perform any actions using parts that are diagnosed as broken.

To guide replanning by diagnostic reasoning, we obtain a replanning domain description from the planning domain description as follows.

For every action constant $A \in \mathcal{A}$, let $A_R \subseteq \mathcal{R}$ be the set of all pairs (r, p) of robots r and their parts p such that r takes part in executing the action A and that $\text{disables}(A_R, A, F)$ holds for some fluent constant $F \in \mathcal{F}$. The replanning domain description $\mathcal{D}_R^{k-t}$ is obtained from $\mathcal{D}^{k-t}$ by adding the following constraints for every time step $i \leq k - t - 1$

$$\leftarrow A(i), \text{broken}(r, p, i) \quad ((r, p) \in A_R). \quad (9)$$

Definition 8: A replanning problem RP is characterized by a tuple $\langle \mathcal{D}_R^{k-t}, s_t, s_g, X_t, k - t \rangle$ where we have the following.

- 1) $\mathcal{D}_R^{k-t}$ is the replanning domain description in ASP.
- 2) $k - t$ is the maximum length of a plan to reach s_g from s_t .

A solution of RP is a sequence $\langle A_0, \dots, A_{m-1} \rangle$ of actions ($m \leq k - t - 1$) (called a *replan*) such that $\mathcal{D}_R^{k-t}$ when combined with the following rules has an answer set:

$$\begin{aligned} &\text{broken}(r, p, 0) \leftarrow ((r, p, i) \in X_t) \\ &\leftarrow \text{not } F_{s_t}(0) \\ &\leftarrow \text{not } F_{s_g}(m) \\ &\leftarrow \text{not } E_{A_i}(i) \quad (i \leq m - 1). \end{aligned} \quad (10)$$

A. Repairs

If there does not exist a solution to the replanning problem RP , then we can consider repairing some of the broken components of the robots before we replan. To find a solution to the replanning problem RP while repairing a set Y of minimum number of broken parts, we replace the constraints (9) in $\mathcal{D}_R^{k-t}$ with the following weak constraints:

$$:\sim A(i), \text{broken}(r, p, i). [1@3, r, p] \quad ((r, p) \in \mathcal{R}).$$

Let us denote the updated description by $\mathcal{D}_{Rfix}^{k-t}$.

Definition 9: Let $RP = \langle \mathcal{D}_R^{k-t}, s_e, s_g, X_t, k - t \rangle$ be a replanning problem. A *replan with repairs* consists of a sequence $P = \langle A_0, \dots, A_{m-1} \rangle$ of actions ($m \leq k - t - 1$) such that $\mathcal{D}_{Rfix}^{k-t}$ when combined with (10) has an answer set Z , and a set

$Y \subset \mathcal{R}$ of parts to be repaired such that $Y = \{(r, p) \in \mathcal{R} : A(i) \in Z, \text{broken}(r, p, i) \in Z, 0 \leq i \leq m-1\}$.

Some broken parts can be repaired more easily or on time, considering the available resources (e.g., consider a planetary rover). Such *a priori* knowledge can be useful to identify which parts are more preferable to repair while replanning. Suppose that atoms of the form $\text{pref}_x(r, p, w)$ where w is a positive integer, express the user's preference of repairing a component p of a robot r considering the available resources. Then, the description $\mathcal{D}_{\text{Rfix}}^{k-t}$ can be obtained from $\mathcal{D}_R^{k-t}$ by replacing the constraints (9) with the following weak constraints:

$:\sim A(i), \text{broken}(r, p, i), \text{pref}_x(r, p, w).[-w@3, r, p]((r, p) \in \mathcal{R})$.

Replanning with this weak constraint will involve repairing a most-preferable min-cardinality set of broken components.

VIII. PLAN EXECUTION MONITORING WITH DIAGNOSTIC REASONING

Based on the methods for hybrid planning (Section IV), diagnostic reasoning and explanation generation (Section V), discrepancy detection under uncertainty and checking for their relevancy (Section VI), and guided replanning and repairs (Section VII), we propose a generic execution monitoring framework that is presented in Algorithm 1.

A. Autonomous Execution Monitoring

Our autonomous execution monitoring algorithm takes as input a robotic domain description $\mathcal{D}$, an initial state s_0 , a goal state s_g , an upper bound k on time steps, a set $\mathcal{R}$ of pairs of robots and their components that may get broken, a relation *disables* that describes which effects of actions are affected if some robots and their components in $\mathcal{R}$ are broken, and a set M of monitored fluents (denoting what is being observed). The algorithm returns success if a goal state is reached and returns a failure report otherwise.

First, Algorithm 1 tries to find a plan P that leads to the goal state (line 4); if such a plan does not exist, P is null, and the algorithm returns failure. Suppose that a plan P exists. Then the algorithm starts running the plan and makes observations over the monitored fluents in M from time to time.

Whenever a new observation becomes available, the algorithm obtains the current values o_t^M of the monitored fluents in M (line 15) and updates the set of observations O_t^M made so far. Then the algorithm infers the full expected state s_e reached from s_0 by the part $P_{<t}$ of the plan executed so far (line 18) and checks whether there is a discrepancy between the expected state s_e and the partially observed current state o_t^M (line 20).

If there exists a discrepancy, then Algorithm 1 generates a full valid state s_v that is consistent with the observations o_t^M and that utilizes knowledge from expected state s_e as much as possible (line 23) and then checks the relevancy of the detected discrepancy by checking whether the rest of the plan $P_{\geq t}$ reaches the goal state s_g from s_v (line 25).

If the discrepancy is found relevant, then Algorithm 1 generates a most-probable min-cardinality diagnosis with explanations X^E (line 28) utilizing the given knowledge prob_b about the

Algorithm 1: Autonomous Execution Monitoring With Diagnostic Reasoning and Repair.

Input: $\mathcal{D}, \mathcal{R}, \text{disables}, s_0, s_g, k, M, \text{prob}_b, \text{pref}_x$.

Output: Return success if the goal state s_g is achieved, return a failure report otherwise.

// Find a plan $P = \langle A_0, \dots, A_{n-1} \rangle$ ($n < k$)

(Definition 1).

1: $P \leftarrow \text{Plan}(\mathcal{D}, s_0, s_g, k)$;

2: **if** P is not null **then**

3: $\text{makespan} = |P|$;

 // Monitor the execution of P .

 // Initially, the set O_0^M of observations and the diagnosis X_0 are empty.

4: $t = 0, O_t^M = \{\}, X_t = \{\}$;

5: **while** $t < \text{makespan}$ **do**

6: Execute A_t ;

7: $t = t + 1$;

 // Observe the monitored fluents M from time to time.

8: **if** a new observation is made at step t **then**

9: $o_t^M \leftarrow$ Obtain the set of current values of M ;

10: $O_t^M \leftarrow$ Add o_t^M to the set of earlier observations O_{t-1}^M ;

 // Infer the expected state s_e at step t , considering the plan $P_{<t} = \langle A_0, \dots, A_{t-1} \rangle$ executed so far and the set $X_{<t} = \cup_{i<t} X_i$ of earlier diagnoses (Definition 4).

11: $s_e \leftarrow \text{Predict}(\mathcal{D}_{\text{diag}}, s_0, P_{<t}, O_t^M, X_{<t})$;

 // Check whether there is a discrepancy between the expected state s_e and the current observations o_t^M (Definition 5).

12: $\text{discrepancy} \leftarrow \text{Discrepancy}(\mathcal{D}_{\text{state}}, s_e, o_t^M)$;

13: **if** *discrepancy* **then**

 // Infer a valid full state s_v from the partially observed current state o_t^M that is closest to the expected state s_e (Definition 6).

14: $s_v \leftarrow \text{ValidState}(\mathcal{D}_{\text{state}}, s_e, o_t^M)$;

 // Check whether the detected discrepancy is relevant to the rest $P_{\geq t} = \langle A_t, \dots, A_{\text{makespan}-1} \rangle$ of the plan (Definition 7).

15: $\text{relevant} \leftarrow$

$\text{Relevancy}(\mathcal{D}_{\text{diag}}, s_e, o_t^M, s_v, P_{\geq t}, s_g, X_{<t})$;

16: **if** *relevant* **then**

 // Find a most-probable min-cardinality diagnosis X^E with an explanation, for the detected relevant discrepancy (Definition 3), with respect to prob_b .

17: $X^E \leftarrow$

$\text{Diagnose}(\mathcal{D}_{\text{diag}}, \mathcal{R}, s_0, P_{<t}, O_t^M, X_{<t}, \text{prob}_b)$;

18: $X_t = \{(r, p, i) | (r, p, A, i) \in X^E\}$;

 // Infer the expected state s_t at step t , considering the plan $P_{<t}$ executed so far and the current diagnosis X_t .

19: $s_t \leftarrow \text{Predict}(\mathcal{D}_{\text{diag}}, s_0, P_{<t}, O_t^M, X_t)$;

 // Replan with a most-preferable min-cardinality set $Y \subset \{(r, p) | (r, p, i) \in X_t\}$ of repairs with respect to pref_x : take s_t as the initial state and $k - t$ as the upper bound on makespan (Definition 9).

```

20:       $P, Y \leftarrow \text{RePlan}(\mathcal{D}_R^{k-t}, s_t, s_g, X_t,$ 
       $k - t, \text{pref}_x);$ 
21:      if  $P$  is not null then
22:           $\text{makespan} = t + |P|;$ 
      // Shift time indices of each action  $A_i$  in  $P$  by  $t$ .
      // Generate a report with explanations and repair
      requests.
23:          Repair the broken parts in  $Y$ ;
24:      else
25:          return Failure;
26:      return Success;
27:  else
28:      return Failure;

```

likelihood of components getting broken. Next, the algorithm infers the current state by computing an expected state s_t at step t reached by the plan $P_{\leq t}$ executed so far and considering the current diagnosis X_t included in X^E . Then, the algorithm continues by replanning at s_t guided by this diagnosis, and possibly repairing a min-cardinality set Y of components (line 33) utilizing the user's preferences and experiences pref_x .

Once all the actions in the plan are executed, the goal is reached with a success.

B. Interactive Execution Monitoring

Interactive execution monitoring is important to involve the expertise of the user in the loop. Thanks to the modular and formal aspects of our execution monitoring framework, we can revise Algorithm 1 easily to make it more interactive.

For instance, instead of generating a diagnosis with explanations and automatically replanning with respect to this diagnosis, the algorithm can generate a set of most-probable min-cardinality diagnoses with explanations and then consult the user as to which one to consider. Meanwhile, the likelihood of each robotic component getting broken is updated based on the selected diagnoses, utilizing the user's expertise.

Instead of generating a single replan P with repairs Y , the algorithm can generate a set of replans with the minimum number of preferable repairs and then consult the user as to which replan to consider. Providing such alternatives is important as the repairs may require different amounts of time and energy, depending on the available resources.

IX. CASE STUDY WITH A COLLABORATIVE SERVICE ROBOTICS DOMAIN

Let us demonstrate the use of our execution monitoring algorithm (Algorithm 1) with a service robotics application, where two bi-manual mobile service robots collaborate to set up a dinner table in a kitchen.

This robotic domain is interesting since 1) it involves both manipulation tasks (e.g., via pick and place actions) and navigation tasks (e.g., via move actions), 2) the executability of manipulation and navigation actions requires feasibility checks (e.g., via reachability checks and motion planning queries), and

3) the locations of the objects and the robots in the kitchen may be unobservable depending on their whereabouts.

A. ASP Representation of the Kitchen Domain

We consider the following set of atoms to describe the domains of variables used in our ASP formulation.

- 1) $\text{rob}(r)$: r is a robot.
- 2) $\text{obj}(o)$: o is an object that the robots can manipulate in the environment.
- 3) $\text{thg}(th)$: th is a thing (i.e., robots and objects).
- 4) $\text{manip}(m)$: m is a (e.g., left or right) manipulator of a robot.
- 5) $\text{robloc}(l)$: l is a location that a robot can be at.
- 6) $\text{objloc}(l)$: l is a possible location for an object.
- 7) $\text{comloc}(l)$: l is a possible location both for an object and for a robot.
- 8) $\text{loc}(l)$: l is a possible location for a thing.
- 9) $\text{time}(i)$: $i \in \{0, \dots, k\}$ where k is a given upper bound on the makespan.
- 10) $\text{atime}(i)$: $i \in \{0, \dots, k-1\}$ where k is a given upper bound on the makespan.

We also denote by $\text{man}(m, r)$ a manipulator m of a robot r .

We consider the following set of fluents and actions.

- 1) $\text{at}(th, l, i)$: Thing th is located at l at time step i .
- 2) $\text{move}(r, l, i)$: The robot r moves to location l at time step i .
- 3) $\text{pickUp}(r, m, o, i)$: The robot r picks up the object o with its manipulator m at time step i .
- 4) $\text{placeOn}(r, m, l, i)$: The robot r places the object at its manipulator m onto a location l at time step i .

First, we define the states (i.e., nodes) of the transition system. Initially, things can be placed arbitrarily:

$$\{\text{at}(th, l, 0)\} \leftarrow \text{thg}(th), \text{loc}(l).$$

At every time step, a thing should be located at some location (existence) but cannot be located at two different locations (uniqueness)

$$\begin{aligned} &\leftarrow \{\text{at}(th, l, i): \text{loc}(l)\} 0, \text{thg}(th), \text{time}(i). \\ &\leftarrow 2\{\text{at}(th, l, i): \text{loc}(l)\}, \text{thg}(th), \text{time}(i). \end{aligned}$$

Moreover, the things should be at their relevant locations

$$\begin{aligned} &\leftarrow \text{at}(o, l, i), \text{obj}(o), \text{loc}(l), \text{not objloc}(l), \text{time}(i). \\ &\leftarrow \text{at}(r, l, i), \text{rob}(r), \text{loc}(l), \text{not robloc}(l), \text{time}(i). \end{aligned}$$

We should also ensure that a robot cannot hold more than one object in its manipulator

$$\leftarrow 2\{\text{at}(o, \text{man}(r, m), i): \text{obj}(o)\}, \text{rob}(r), \text{manip}(m), \text{time}(i).$$

We define the transitions (i.e., edges) of the transition system. Consider the edges that describe nonoccurrences of actions. We express that a thing remains where it is, unless some action causes a change of its location as follows:

$$\{\text{at}(th, l, i+1)\} \leftarrow \text{at}(th, l, i), \text{thg}(th), \text{loc}(l), \text{atime}(i).$$

The other edges describe occurrences of actions. Actions may occur at any time

$$\begin{aligned} \{move(r, l, t)\} &\leftarrow rob(r), robloc(l), atime(i). \\ \{pickUp(r, m, o, i)\} &\leftarrow rob(r), manip(m), obj(o), atime(i). \\ \{placeOn(r, m, l, i)\} &\leftarrow rob(r), manip(m), objloc(l), atime(i). \end{aligned}$$

However, an action cannot occur at a state that does not satisfy its preconditions. For instance, a robot r cannot move to a location l if it is already there

$$\leftarrow move(r, l, i), at(r, l, i), rob(r), robloc(l), atime(i).$$

A robot r cannot pick up an object o if they are at different locations, if another robot $r1$ is holding o but the robot is at a different location, or if the object o is on the table but r is not around the table

$$\begin{aligned} &\leftarrow pickUp(r, m, o, i), at(o, l, i), at(r, l1, i), rob(r), obj(o), \\ &\quad manip(m), robloc(l1), comloc(l), atime(i) \quad (l \neq l1). \\ &\leftarrow pickUp(r, m, o, i), at(o, man(r1, m1), i), at(r, l, i), \\ &\quad at(r1, l1, i), rob(r), rob(r1), manip(m), manip(m1), obj(o), \\ &\quad robloc(l), robloc(l1), atime(i) \quad (l \neq l1). \\ &\leftarrow pickUp(r, m, o, i), \{at(r, TableLeft, i); at(r, TableRight, i)\}0, \\ &\quad at(o, Table, i), rob(r), manip(m), obj(o), atime(i). \end{aligned}$$

Similarly, a robot r cannot place an object onto some location l with its manipulator m if the manipulator is not holding any object, if the robot r is not at l , if $l=man(r1, m1)$ and the robots r and $r1$ are not at the same location, or if $l=Table$ but the robot r is not around the table

$$\begin{aligned} &\leftarrow placeOn(r, m, l, i), \{at(o, man(r, m), i):obj(o)\}0, rob(r), \\ &\quad manip(m), objloc(l), atime(i). \\ &\leftarrow placeOn(r, m, l, i), at(r, l1, i), rob(r), manip(m), \\ &\quad comloc(l), robloc(l1), atime(i) \quad (l \neq l1). \\ &\leftarrow placeOn(r, m, man(r1, m1), i), at(r1, l1, t), at(r, l, t), \\ &\quad rob(r), rob(r1), manip(m), manip(m1), robloc(l), \\ &\quad robloc(l1), atime(i) \quad (l \neq l1). \\ &\leftarrow placeOn(r, m, Table, i), rob(r), manip(m), atime(i), \\ &\quad \{at(r, TableLeft, i); at(r, TableRight, i)\}0. \end{aligned}$$

Furthermore, we consider some nonconcurrency constraints to ensure that navigation and manipulation actions do not occur at the same time

$$\begin{aligned} &\leftarrow move(r, l, i), pickUp(r, m, o, i), rob(r), robloc(l), \\ &\quad manip(m), obj(o), atime(i). \\ &\leftarrow move(r, l, i), placeOn(r, m, l1, i), rob(r), manip(m), \\ &\quad robloc(l), objloc(l1), atime(i). \end{aligned}$$

Since robots are operating in a continuous space, for the feasibility of actions, we embed the relevant low-level feasibility checks into the domain description. For instance, a robot r cannot move from a location l to another location $l1$ if there is no collision-free trajectory that it can follow. A robot r cannot pick up an object o if the robot cannot reach and grasp the object without any collisions. Similarly, a robot r cannot place an object onto a location if the robot cannot reach the location or place the

object without any collisions

$$\begin{aligned} &\leftarrow at(r, l1, i), move(r, l, i), not \&moveIsFeasible[r, l, l1](), \\ &\quad rob(r), robloc(l), robloc(l1), atime(i). \\ &\leftarrow at(r, l, i), pickUp(r, m, o, i), rob(r), objloc(l), manip(m), \\ &\quad obj(o), not \&pickUpsIsFeasible[r, l, o, m](), atime(i). \\ &\leftarrow at(r, l, i), placeOn(r, m, l, i), rob(r), objloc(l), manip(m), \\ &\quad not \&placeOnIsFeasible[r, l, m](), atime(i). \end{aligned} \quad (11)$$

Let us now define what changes after execution of an action. For instance, when a robot r moves to a location l at step i , its location becomes l at the next time step

$$at(r, l, i+1) \leftarrow move(r, l, i), rob(r), robloc(l), atime(i).$$

When a robot r picks up an object o with its manipulator, the location of object becomes the manipulator at the next time step

$$at(o, man(r, m), i+1) \leftarrow pickUp(r, m, o, i), rob(r), manip(m), obj(o), atime(i).$$

When a robot r places the object o that it is holding with its manipulator to a location l , the location of the object becomes l at the next time step

$$at(o, l, i+1) \leftarrow placeOn(r, m, l, i), at(o, man(r, m), i), rob(r), objloc(l), manip(m), atime(i).$$

B. Implementation of the Feasibility Checks

We have utilized several feasibility checks for implementing the external atoms in (11) and to ensure the feasibility of robotic actions in the kitchen domain. These checks are implemented using OPENRAVE 0.9 [53]. Self-collisions and environmental collisions are checked by using OPEN DYNAMICS ENGINE (ODE).

The navigation action $move(r, l, t)$ requires the existence of a collision-free path between the current robot position and the destination location l . The existence of such a path is checked with the external atom $\&moveIsFeasible[r, l, l1]()$ as follows. First, candidate robot base positions are sampled in the destination area. For example, if the robot needs to go to *ShelfA*, the external atom samples some candidate robot base positions around the shelf. Next, it calls the Bi-RRT motion planner of OPENRAVE to check the existence of a collision-free path to each sampled destination position.

The external atom $\&pickUpsIsFeasible[r, l, o, m]()$ performs the relevant feasibility checks for the executability of a pick action as follows. First, it identifies possible collision-free grasps of the object o , using the *graspability* database of OPENRAVE for identifying possible grasps. Then, for each valid grasp, it checks if the grasp is possible by the manipulator m , using the *inversekinematics*, *convexdecomposition*, *inversereachability*, and *kinematicreachability* databases provided by OPENRAVE. It also checks if a collision-free trajectory exists for the manipulator to reach the object location l , using the RRT-Connect motion planner of OPENRAVE and the collision checker of ODE.

The external atom $\&placeOnIsFeasible[r, l, m]()$ performs the relevant feasibility checks for the executability of a place action as follows. For each manipulator m with an object in hand to be placed on location l , a target location is sampled

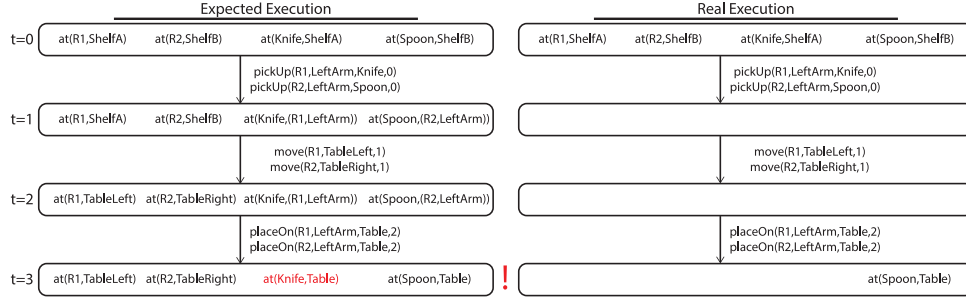


Fig. 1. Expected execution (with expected world states) and a real execution (with observations) of the initial plan P at time step $t = 0$.

and candidate collision-free positions are identified. For each possible position, the *inversereachability* database provided by OPENRAVE is called along with the RRT-Connect motion planner of OPENRAVE and the collision checker of ODE to get a valid robot base location and manipulator trajectory.

If the feasibility checks do not return with a valid solution within a given time threshold (or within a given number of samples), these checks are assumed to be infeasible. Note that a valid solution may still exist and may be computed, if sufficiently large amount of time is allocated for these checks (since the underlying motion planning algorithms are probabilistically complete).

C. Execution Monitoring of a Plan: A Sample Scenario

We introduce an illustrative example in the kitchen domain to show how our execution monitoring algorithm (Algorithm 1) can be used autonomously or interactively, emphasizing the differences among the three diagnostic reasoning approaches explained in Section V-B.

Consider the kitchen domain with two robots ($R1$ and $R2$), two distinct objects ($Knife$ and $Spoon$), and three distinct locations ($ShelfA$, $ShelfB$, and $Table$). Assume that the world is partially observable: Only the objects located on the table can be observed. Let us also assume that the bases of $R1$ and $R2$ are more likely to get broken compared to other components. Initially, at s_0 , $Table$ is empty, $Knife$ is on $ShelfA$, and $R1$ is near $ShelfA$, while $Spoon$ is on $ShelfB$ and $R2$ is near $ShelfB$. At the goal state s_g , both $Knife$ and $Spoon$ are on $Table$.

The algorithm solves this planning problem instances by utilizing the kitchen domain description above

$P = \langle \text{pickUp}(R1, \text{LeftArm}, \text{Knife}, 0), \text{pickUp}(R2, \text{LeftArm}, \text{Spoon}, 0), \text{move}(R1, \text{TableLeft}, 1), \text{move}(R2, \text{TableRight}, 1), \text{placeOn}(R1, \text{LeftArm}, \text{Table}, 2), \text{placeOn}(R2, \text{LeftArm}, \text{Table}, 2) \rangle$.

Then, the robots start executing the plan P . The left column of Fig. 1 illustrates the expected world states during a successful execution of the plan P , while the right column shows the observations made during a real execution of P . Fig. 2 shows the dynamic simulations for both the expected and real executions of the plan P , for better understandability. A video of plan execution under different diagnostic reasoning approaches is available at http://cogrobo.sabanciuniv.edu/diagnostic_reasoning.mp4.

While P is being executed, suppose that an observation o_t^M has been performed at time step $t = 3$: Only $Spoon$ is on

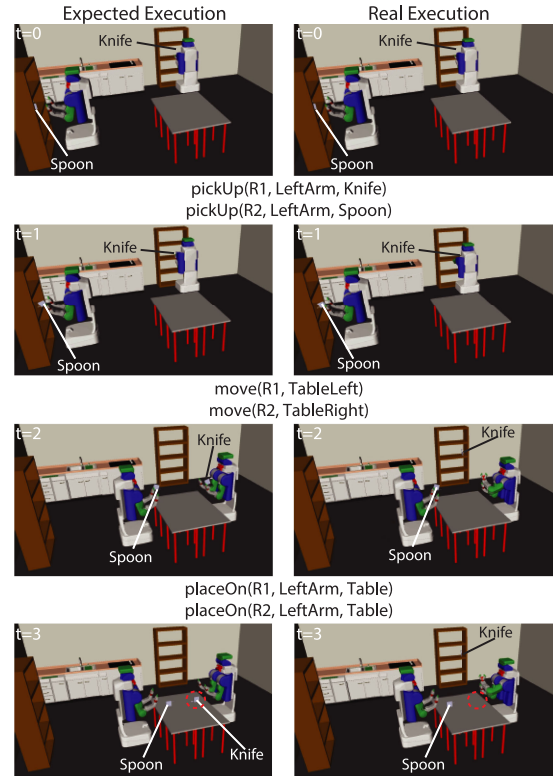


Fig. 2. Dynamic simulations for the expected and the real executions of the initial plan P at time step $t = 0$ presented in Fig. 1.

$Table$. The expected state s_e at $t = 3$ is predicted as follows: $at(R1, TableLeft)$, $at(R2, TableRight)$, $at(Knife, Table)$, and $at(Spoon, Table)$. Since $Knife$ is not observed to be on $Table$ as expected, there is a discrepancy between s_e and o_3^M .

Note that once a full valid state s_v is estimated (where $Spoon$ is on $Table$ but $Knife$ is not on $Table$), this discrepancy is found relevant for the rest $P_{\geq} = \emptyset$ of the plan: Executing $P_{\geq}$ at s_v does not lead the plan to the goal state s_g .

Then, according to the autonomous mode of the algorithm, considering that the bases are more likely to get broken, the following most-probable min-cardinality diagnosis is computed for the relevant discrepancy detected at time step $t = 3$

$$X_3 = \{(R1, Base, 1)\}$$

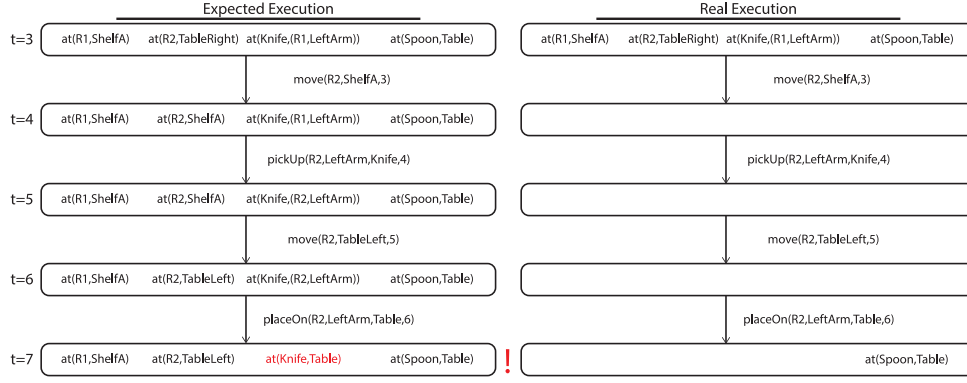


Fig. 3. Expected execution (with expected world states) and a real execution (with observations) of the replan P computed at time step $t = 3$ considering the diagnosis $X_t = \{(R1, Base, 1)\}$.

with an explanation: “Base of $R1$ got broken at time step $t = 1$; so $R1$ could not move to *TableLeft* at time step $t = 1$ as expected.” Note that since this is the first diagnosis, all three diagnostic reasoning approaches (i.e., augmented, reset, and revised) may generate the same hypothesis.

In the interactive mode of this algorithm, the following set of three min-cardinality diagnoses is presented to the user

$$\{\{(R1, Base, 1)\}, \{(R1, LeftArm, 0)\}, \{(R1, LeftArm, 2)\}\}$$

with explanations:

- 1) The base of $R1$ got broken at time step $t = 1$; so $R1$ could not move to *TableLeft* at time step $t = 1$ as expected.
- 2) The left arm of $R1$ got broken at time step $t = 0$; so $R1$ could not pick up *Knife* at time step $t = 0$ as expected.
- 3) The left arm of $R1$ got broken at time step $t = 2$; so $R1$ could not place *Knife* on *Table* at time step $t = 2$ as expected.

Then, the user picks one of them for the algorithm to proceed with replanning. For instance, considering that the bases are more likely to get broken, the user might pick the first diagnosis to proceed: $X_3 = \{(R1, Base, 1)\}$; or considering that the left arm of $R1$ often breaks down early in a plan, the user may pick the second diagnosis: $X_3 = \{(R1, LeftArm, 0)\}$.

Suppose that the user picks $X_3 = \{(R1, Base, 1)\}$. With this diagnosis with explanation, the execution monitoring algorithm predicts the current expected state s_t as follows: *at(R1, ShelfA)*, *at(R2, TableRight)*, *at(Knife, R1Hand)*, and *at(Spoon, Table)*. After that, the algorithm replans to reach the goal state s_g from s_3 as follows:

$$P = \langle \text{move}(R2, \text{ShelfA}, 3), \text{pickUp}(R2, \text{LeftArm}, \text{Knife}, 4), \text{move}(R2, \text{TableLeft}, 5), \text{placeOn}(R2, \text{LeftArm}, \text{Table}, 6) \rangle.$$

Then, the robots start executing the new plan P at time step $t = 3$. The expected execution and a real execution of P are shown in the left and right columns of Fig. 3, respectively, while Fig. 4 illustrates their dynamic simulations.

Assume that the next observation o_t^M takes place at time step $t = 7$: Only *Spoon* is on *Table*. The expected state s_e at $t = 7$ is predicted as follows: *at(R1, ShelfA)*, *at(R2, TableLeft)*,

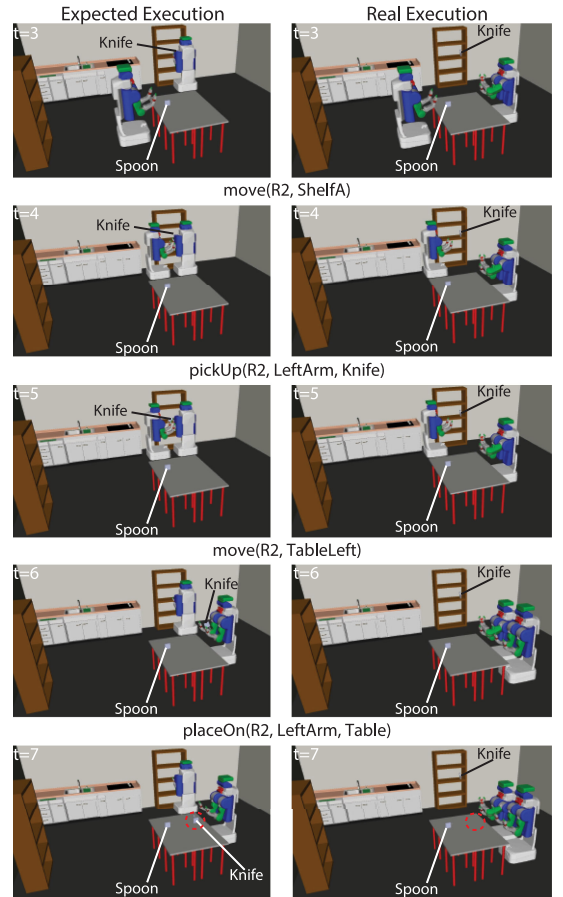


Fig. 4. Dynamic simulations for the expected and the real executions of the replan P at time step $t = 3$ presented in Fig. 3.

at(Knife, Table), and *at(Spoon, Table)*. Since *Knife* is not observed to be on *Table* as expected, there is a discrepancy between s_e and o_t^M . Furthermore, this discrepancy is relevant as the goal state is not reached by the execution of the rest of the plan.

Then, according to the interactive mode of the algorithm, a set of min-cardinality diagnoses is computed so that the user can pick the most-probable one.

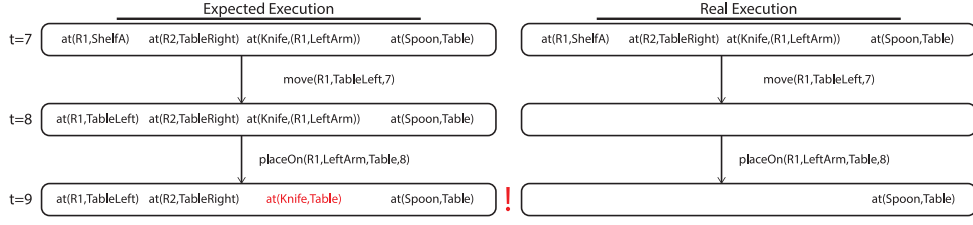


Fig. 5. Expected execution (with expected world states) and a real execution (with observations) of the replan P computed at time step $t = 7$ considering the reset diagnosis $X_t = \{(R2, Base, 3)\}$.

Augmented diagnoses: For instance, a set of four min-cardinality augmented diagnoses can be generated based on the current observation o_7^M , while imposing the previous diagnoses $X_{<7} = X_3 = \{(R1, Base, 1)\}$

$$\begin{aligned} &\{ \{(R1, Base, 1), (R2, Base, 3)\}, \{(R1, Base, 1), (R2, Base, 5)\}, \\ &\{(R1, Base, 1), (R2, LeftArm, 4)\}, \{(R1, Base, 1), \\ &(R2, LeftArm, 6)\} \}. \end{aligned}$$

Once these augmented diagnoses with their explanations are presented to the user, assume that the user selects the min-cardinality diagnosis $X_7 = \{(R1, Base, 1), (R2, Base, 3)\}$ with the following explanation: “Base of $R1$ got broken at time step $t = 1$, so $R1$ could not move to *TableLeft* at time step $t = 1$ as expected; and since *Base* of $R2$ got broken at time step $t = 3$, $R2$ could not move to *ShelfA* at time step $t = 3$ as expected.”

However, since the bases of both robots are diagnosed as broken, the robots cannot move around, and, thus, replanning fails. This example shows that although augmented diagnoses generously maintain the previous diagnoses, they put more constraints on replanning (as more number of parts are assumed to be broken) making it harder to find a replan.

Reset diagnoses: Alternatively, a set of four min-cardinality reset diagnoses can be generated based on the current observation o_7^M but without considering any of the previous diagnoses:

$$\begin{aligned} &\{ \{(R2, Base, 3)\}, \{(R2, Base, 5)\}, \{(R2, LeftArm, 4)\}, \\ &\{(R2, LeftArm, 6)\} \}. \end{aligned}$$

Suppose that the user selects the first min-cardinality diagnosis $X_7 = \{(R2, Base, 3)\}$ with explanation: “Base of $R2$ is broken; so $R2$ could not move to *ShelfA* at time step 3 as expected.”

Considering this diagnosis, the algorithm predicts the expected current state s_t for time step $t = 7$ and replans to reach the goal state s_g from s_7 as follows:

$$P = \langle \text{move}(R1, \text{TableLeft}, 7), \text{placeOn}(R1, \text{LeftArm}, \text{Table}, 8) \rangle.$$

The expected execution and a real execution of the new plan P are shown in Fig. 5, respectively.

Suppose that, at time step $t = 9$, another observation o_t^M is made: Only *Spoon* is on *Table*. The expected state s_e at $t = 9$ is predicted as follows: $\text{at}(R1, \text{TableLeft})$, $\text{at}(R2, \text{TableRight})$, $\text{at}(\text{Knife}, \text{Table})$, and $\text{at}(\text{Spoon}, \text{Table})$. There is a discrepancy between s_e and o_9^M , and it is relevant for the rest of the plan.

The following set of two min-cardinality reset diagnoses can be generated based on the current observation o_t^M :

$$\{ \{(R1, Base, 7)\}, \{(R1, LeftArm, 8)\} \}.$$

Assume that the user picks the reset diagnosis $X_9 = \{(R1, Base, 7)\}$. Note that the diagnosed broken component is identical with that of the first diagnosis X_3 . This example shows that reset diagnoses may unnecessarily repeat themselves in terms of broken components, as they forget the previous diagnoses and the observations. As a result, replanning guided with a reset diagnosis may keep generating the same plan that may not reach the goal.

Revised diagnosis: Alternatively, a set of two min-cardinality revised diagnoses can be generated for the detected relevant discrepancy at time step $t = 7$, considering both the current observation o_7^M and the previous observation o_3^M (i.e., $O_7^M = \{o_3^M, o_7^M\}$) while updating the previous diagnoses as follows:

$$\{ \{(R1, LeftArm, 0)\}, \{(R1, LeftArm, 2)\} \}.$$

Assume that the user selects the revised diagnosis $X_7 = \{(R1, LeftArm, 0)\}$ with its explanation: “LeftArm of $R1$ got broken at time step $t = 0$; so $R1$ could not pick up *Knife* at time step $t = 0$ as expected.”

Once the algorithm predicts the expected current state s_7 with respect to this diagnosis, it proceeds with replanning

$$P = \langle \text{move}(R2, \text{ShelfA}, 7), \text{pickUp}(R2, \text{LeftArm}, \text{Knife}, 8), \\ \text{move}(R2, \text{TableLeft}, 9), \text{placeOn}(R2, \text{LeftArm}, \text{Table}, 10) \rangle.$$

The expected execution and a successful real execution of this new plan P at time step $t = 7$ are shown in the left and right columns of Fig. 6, respectively. Fig. 7 illustrates their dynamic simulations.

Repairs: Sometimes the generated diagnoses prevent the algorithm to find a replan as we have observed with the augmented diagnosis computed for the relevant discrepancy detected at time step $t = 7$: Since the bases of both robots are diagnosed as broken, the robots cannot move around, and, thus, replanning fails. In such cases, the algorithm may repair a minimum number of broken parts while replanning.

Suppose that the algorithm computes the following set of all min-cardinality diagnoses, and none of them leads to a replan:

$$\begin{aligned} &\{ \{(R1, Base, 1), (R1, LeftArm, 3)\}, \{(R1, Base, 1), (R2, Base, 3)\}, \\ &\{(R1, LeftArm, 3), (R2, RightArm, 5)\}, \{(R1, LeftArm, 3), \\ &(R2, Base, 3)\}, \{(R2, RightArm, 5), (R1, Base, 1)\} \}. \end{aligned}$$

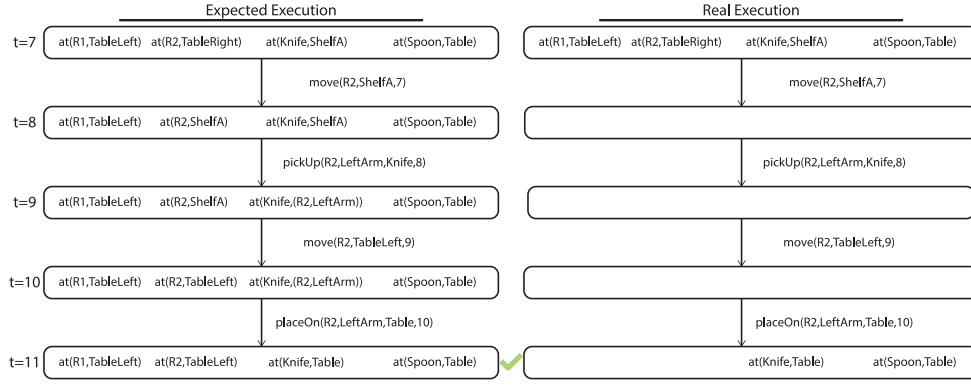


Fig. 6. Expected and the real executions of the replan P computed at time step $t = 7$, with respect to the revised diagnosis $X_7 = \{(R2, Base, 3)\}$.

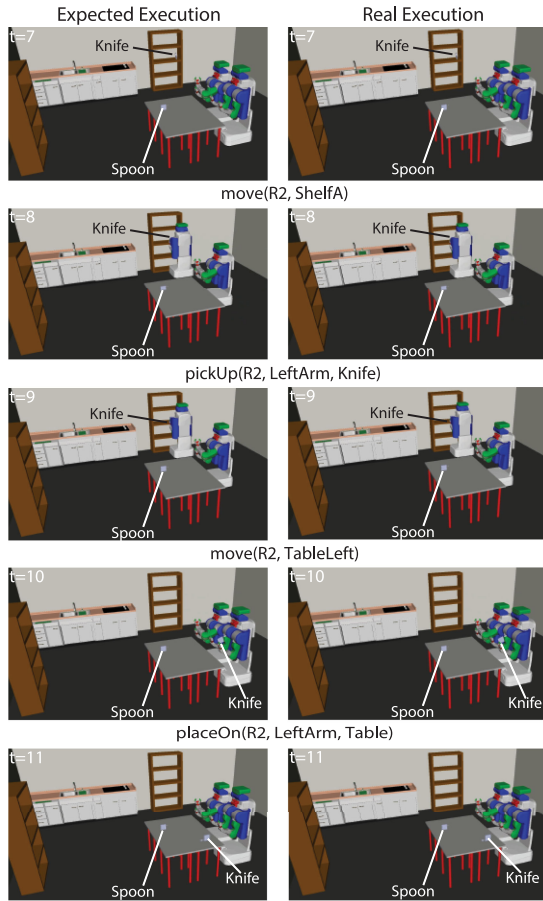


Fig. 7. Dynamic simulations for the expected and the real executions of the replan presented in Fig. 6, where the plan is completed successfully.

The algorithm may consult the user about the preference over repairing these parts. Suppose that the user knows that the parts of robot $R2$ take usually more time to repair since it is an old model. Alternatively, the user may check all the computed diagnoses above and identify the parts that more often appear in diagnoses. Then, this information ($pref_x$) can be utilized while replanning as explained in Section VII-A. For instance, the following replan is computed with repairs

$\{(R1, Base), (R1, LeftArm)\}$:

$P = \langle move(R1, ShelfA, 5), pickUp(R1, LeftArm, Knife, 6), move(R1, TableLeft, 7), placeOn(R1, LeftArm, Table, 8) \rangle$.

Since the parts $(R1, Base)$ and $(R1, LeftArm)$ are repaired, they are utilized in the new plan.

X. EXPERIMENTAL EVALUATION

We have performed two groups of experiments. First, we have evaluated the computational performance of our execution monitoring algorithm (Algorithm 1) with different replanning approaches.

- 1) *No guidance, full observability*: Whenever a relevant discrepancy is detected, the execution monitoring algorithm does not perform diagnostic reasoning and replans from the current state under the full observability assumption, considering the low-level feasibility checks.
- 2) *Guidance via diagnosis, partial observability*: Whenever a relevant discrepancy is detected, the execution monitoring algorithm computes a diagnosis for it (e.g., which broken components may cause this discrepancy). Then, the algorithm replans guided by this diagnosis. For instance, if a robotic component is diagnosed to be broken, then the algorithm does not allow the use of this broken part while replanning.

Second, we have experimented with three different approaches to diagnostic reasoning to better understand their usefulness within the execution monitoring algorithm.

- 1) *Augmented diagnosis*: A diagnosis is generated about the broken parts based on the current observation, while imposing the validity of all of the previous hypotheses.
- 2) *Reset diagnosis*: A diagnosis is generated about the broken parts based on the current observation, without considering any of the previous hypotheses.
- 3) *Revised diagnosis*: A diagnosis is generated about the broken parts based on the current and the previous observations, while updating the previous hypothesis as necessary.

In these experiments, we have evaluated the validity of the following statements.

- S1 Utilizing diagnostic reasoning in the execution monitoring algorithm increases the success of reaching the goal state while decreasing the number of replannings.
- S2 Utilizing revised diagnosis (instead of reset or augmented diagnosis) in the execution monitoring algorithm increases the success of reaching the goal state while decreasing the number of replannings.
- S3 The execution monitoring algorithm equipped with revised diagnosis (instead of reset or augmented diagnosis) is more scalable in terms of computation time as the size of a diagnosis increases.

A. Experimental Setup

We have considered the kitchen domain (Section IX-A) for our experimental evaluations. Recall that, in this domain, multiple bimanual mobile service robots collaboratively set up a dinner table in a kitchen. The goal for the robots is to place all objects initially scattered around the kitchen on the table.

Each benchmark instance $I = \langle s_0, s_g, n, B \rangle$ is described by an initial state s_0 , a goal state s_g , a makespan n ($n < k$), and a set B of pairs (b, i) of a number b of broken robotic parts and a time step i . Each pair describes how many parts get broken and at what time step. No physical plan execution takes place during the evaluations. Instead, for each benchmark instance $I = \langle s_0, s_g, n, B \rangle$, we have the following.

- 1) An initial plan $P_I = \langle A_0, \dots, A_{n-1} \rangle$ ($n < k$) is computed to reach s_g from s_0 in k steps.
- 2) A set X_I of tuples (r, p, i) are randomly generated respecting B (i.e., for each (b, i) in B , the number of tuples (r, p, i) in X_I is exactly b) such that every component (r, p) plays a role in the actions A_i of the plan P (i.e., for every (r, p, i) in X_I , $\text{disables}(X, A_i, F)$ where $(r, p, i) \in X$).

Then, the “real world” states expected during the execution of this plan P_I are computed by a dynamic simulation considering X_I . Along these lines, the observations over the monitored fluents are extracted from these “real world” states. The upper bound k on the total plan length is set to 60.

All experiments have been performed on a Windows server with six 3.5 GHz Intel Xeon E5-1650 v3 CPU cores and 8-GB memory. Reasoning problems have been solved using CLINGO 4.5.4 with multithreading (limited to four threads). We have considered a timeout of 100 s per planning instance, with the anytime option of CLINGO.

B. Evaluation Criteria

We have evaluated statements S1, S2, and S3 from the following perspectives: success rate and computational efficiency.

For every instance $I = \langle s_0, s_g, n, B \rangle$ and an initial plan P_I , we say that I is *successful* if the goal state s_g is reached by execution monitoring of P_I by our algorithm without exceeding the total plan length k . Accordingly, the success rate is defined as follows:

$$\text{Success Rate} = \frac{\# \text{ successful instances}}{\# \text{ all instances}} \times 100.$$

For every instance $I = \langle s_0, s_g, n, B \rangle$, with an initial plan P_I and a set X_I of broken components, we quantify the computational efficiency of our algorithm by means of the following measures:

- 1) *# replannings*: the number of replannings performed during execution monitoring;
- 2) *total plan length*: the total length of the final plan;
- 3) *diagnosis time*: CPU time in seconds to perform all diagnostic reasoning tasks during execution monitoring;
- 4) *replanning time*: CPU time in seconds, to perform all replannings during execution monitoring;
- 5) *diagnosis accuracy*: the accuracy of the generated diagnosis X_t with respect to X_I .

Note that, during plan execution monitoring, replanning guided by a diagnosis may lead to the goal state even though the diagnosis is incorrect with respect to what is broken. However, proper identification of the broken robotic parts is desirable such that they can be repaired before the next plan execution. To evaluate the accuracy of the diagnoses generated by different diagnostic reasoning approaches, we have introduced a metric called *diagnosis accuracy*.

The diagnosis accuracy for *revised diagnosis* is defined relative to the last revised diagnosis X_t made by the execution monitoring algorithm

$$\text{Accuracy}_{\text{Revised}} = \frac{|X_t \cap X_I|}{\max(|X_I|, |X_t|)} \times 100. \quad (12)$$

Here, the last diagnosis is considered since it utilizes all of the previous observations as well as the current observations. For instance, assume that

$$\begin{aligned} X_I &= \{(R1, \text{LeftArm}, 3), (R2, \text{RightArm}, 5), (R3, \text{Base}, 9)\}, \\ X_3 &= \{(R1, \text{LeftArm}, 3), (R1, \text{RightArm}, 2), (R3, \text{Base}, 9)\}. \end{aligned}$$

Then the diagnosis accuracy is 66.67%.

The diagnosis accuracy for *augmented diagnosis* is defined in the same way

$$\text{Accuracy}_{\text{Augmented}} = \text{Accuracy}_{\text{Revised}}.$$

The diagnosis accuracy for *reset diagnosis* is defined relative to the all reset diagnoses X_i made by the execution monitoring algorithm

$$\text{Accuracy}_{\text{Reset}} = \frac{|X_I \cap \bigcup X_i|}{\max(|X_I|, |\bigcup X_i|)} \times 100. \quad (13)$$

Here, all reset diagnoses are considered since they are computed independently from each other. For instance, assume that

$$\begin{aligned} X_I &= \{(R1, \text{LeftArm}, 3), (R2, \text{RightArm}, 5), (R3, \text{Base}, 9)\}, \\ X_1 &= \{(R1, \text{LeftArm}, 3)\}, \\ X_2 &= \{(R2, \text{LeftArm}, 4)\}, \\ X_3 &= \{(R1, \text{LeftArm}, 3), (R2, \text{RightArm}, 5), (R2, \text{Base}, 8)\}. \end{aligned}$$

Then, since

$$\bigcup X_i = \{(R1, \text{LeftArm}, 3), (R2, \text{LeftArm}, 4), (R2, \text{RightArm}, 5), (R2, \text{Base}, 8)\}$$

the diagnosis accuracy is 50%.

TABLE I
REPLANNING WITH NO GUIDANCE (FULL OBSERVABILITY) VS. REPLANNING GUIDED BY REVISED DIAGNOSIS (PARTIAL OBSERVABILITY)

Number of broken parts	No Guidance			Guidance via Revised Diagnosis		
	# Replannings	Total plan length	Success rate [%]	# Replannings	Total plan length	Success rate [%]
2	8.48 $\pm$ 2.29	26.44 $\pm$ 5.50	80	3.52 $\pm$ 0.92	17.76 $\pm$ 2.77	92
3	14.12 $\pm$ 3.37	35.96 $\pm$ 7.09	72	4.04 $\pm$ 1.30	22.48 $\pm$ 6.46	80

TABLE II
SCALABILITY ANALYSIS WITH DIFFERENT NUMBER OF BROKEN PARTS (COMMONLY SOLVED INSTANCES): *REVISED* VS. *RESET*

	2 Broken Parts		3 Broken Parts		4 Broken Parts		5 Broken Parts		6 Broken Parts	
	Reset	Revised	Reset	Revised	Reset	Revised	Reset	Revised	Reset	Revised
# Replannings	3.52 (1.47)	2.76 (0.94)	4.75 (2.23)	3.75 (1.12)	6.84 (1.77)	5.69 (1.97)	9.28 (2.42)	5.71 (1.79)	6.75 (3.30)	9.75 (0.50)
Total plan length	26.09 (4.88)	24.85 (4.75)	29.87 (7.54)	27.75 (4.53)	39.23 (5.54)	36.76 (6.52)	40.57 (6.60)	36.85 (6.54)	41.00 (8.36)	51.50 (7.32)
Diagnosis time [s]	7.65 (5.23)	4.13 (1.54)	11.86 (9.75)	7.78 (3.16)	20.38 (7.91)	13.61 (4.14)	30.64 (12.76)	21.72 (6.98)	21.34 (15.52)	40.76 (9.55)
Replanning time [s]	67.72 (22.77)	64.94 (28.14)	91.83 (28.49)	81.04 (24.20)	128.71 (35.13)	118.91 (48.53)	133.60 (31.87)	104.69 (16.92)	139.86 (62.35)	169.18 (54.19)
Diagnosis accuracy [%]	65.39 (18.88)	90.40 (20.11)	66.66 (22.92)	89.56 (15.93)	59.94 (13.02)	86.53 (19.40)	63.57 (7.70)	82.85 (17.99)	59.71 (20.97)	74.95 (9.62)
# Successful instances	22	22	18	22	13	21	8	19	5	17
# Common successful instances	21		16		13		7		4	

C. Evaluating Replanning Approaches

To test the validity of S1, we have investigated the usefulness of two approaches to replanning, *No guidance with full observability* and *Guidance via diagnosis with partial observability*, by performing a set of experiments in the kitchen domain with two robots and ten objects. We have generated 25 instances where two robotic parts are broken at different times. Similarly, another 25 instances have been generated where 3 robotic parts are broken. We have run our algorithm for each instance, as described in Section X-A, and reported in Table I the average and the standard deviation (in parenthesis, over 25 instances) of the evaluation metrics described in Section X-B.

Several interesting observations about these two replanning approaches can be made over the following results.

- 1) In terms of the success rate, replanning guided by diagnosis outperforms replanning with no guidance.
- 2) For each number of broken parts, the average number of replannings with guided replanning is much lower.
- 3) The total plan length decreases when replanning is guided by diagnosis. This decrease becomes more significant as the number of broken parts increases.

These results are expected since replanning with no guidance does not consider causes of discrepancies and may try to use the broken robotic components in replans, resulting in new discrepancies. Without guidance, the number of replannings and the total plan length increase with a great margin.

We need to emphasize that even though replanning with no guidance is provided with full observability while guided replanning is performed under partial observability, the guided replanning performs significantly better than replanning with no guidance in terms of the success rate, the number of replannings, and the total plan length.

These results also confirm our statement S1: diagnosis is useful for execution monitoring.

D. Evaluating Diagnostic Reasoning Approaches

To test the validity of S2 and S3, we have evaluated three diagnostic reasoning approaches, *augmented diagnosis*, *reset diagnosis*, and *revised diagnosis*, in terms of the number of replannings, plan length, and CPU time.

1) *Scalability Analysis With Respect to the Number of Broken Robot Components*: We have performed a set of experiments in the kitchen domain with 4 robots and 20 objects. We have generated 25 instances where 2 (respectively 3, 4, 5, and 6) robotic parts are broken at different times. We have run our algorithm three times for each instance, as described in Section X-A, and reported in Table II the average and the standard deviation (over 25 instances, 3 runs) of the evaluation metrics described in Section X-B. The results of these experiments are also presented in Fig. 8(a).

Fig. 8(a) presents the success rates of different diagnostic reasoning approaches, for each number of broken parts in terms of reaching the goal state. According to these results, we have the following.

- 1) Increasing the number of broken parts reduces the success rate in general.
- 2) The success rate of *augmented diagnosis* is quite low compared to the other two diagnostic reasoning approaches. For instance, with six broken parts, *augmented diagnosis* can solve only 4% of the 25 instances, whereas *reset diagnosis* can solve 20% of the instances and *revised diagnosis* can solve almost 68% of the instances.

The first observation above can be explained as follows: Increasing the number of broken parts increases the number of possible most-probable min-cardinality diagnoses and, thus, decreases the likelihood of selecting a correct diagnosis.

The second observation above can be explained as follows. When a relevant discrepancy is detected, *augmented diagnosis* generates a hypothesis about the broken parts based on the current observations, while imposing the validity of the previous diagnoses even though they might not be correct. Therefore, an augmented diagnosis grows each time a relevant discrepancy is detected during plan execution monitoring: larger diagnosis puts more constraints on replanning and, thus, reduces the success rate. *Reset diagnosis* generates a hypothesis for a detected relevant discrepancy based on the current observations but without relying on the previous assumptions or observations. Therefore, sometimes incorrect diagnoses can be made repeatedly, leading to incorrect replans, and reducing the success rate but not as much as *augmented diagnosis*. *Revised diagnosis*, on the other hand, considers the current observation and all of the previous

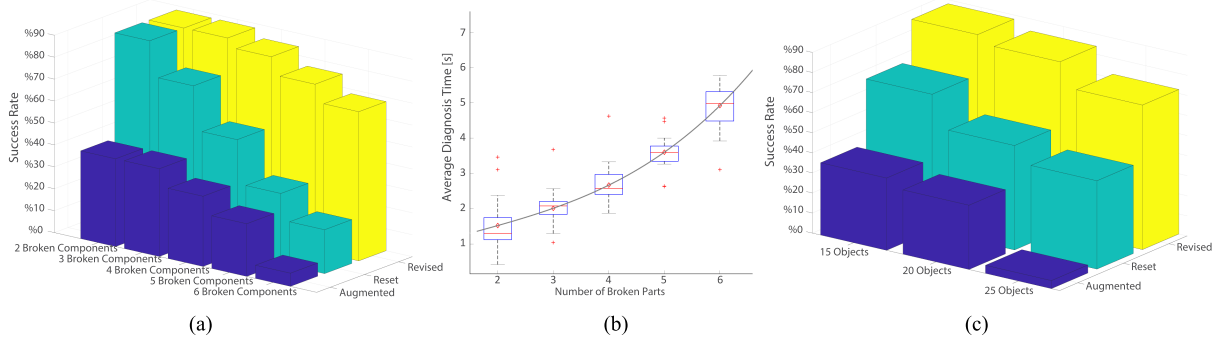


Fig. 8. (a) Effect of number of broken parts on the success rates of different diagnosis approaches. (b) Average CPU time consumption for different number of broken parts during revised diagnosis. (c) Effect of number of objects on the success rate of different diagnosis approaches.

TABLE III
SCALABILITY ANALYSIS WITH DIFFERENT NUMBER OF BROKEN PARTS (ALL INSTANCES): *REVISED* VS. *RESET*

	2 Broken Parts		3 Broken Parts		4 Broken Parts		5 Broken Parts		6 Broken Parts	
	Reset	Revised	Reset	Revised	Reset	Revised	Reset	Revised	Reset	Revised
# Replannings	4.32 (3.86)	2.80 (0.91)	8.72 (6.63)	3.84 (1.14)	8.96 (4.65)	5.60 (1.89)	14.20 (10.06)	5.88 (2.55)	16.20 (7.87)	6.96 (2.54)
Total plan length	27.88 (8.31)	25.32 (4.62)	39.76 (14.84)	29.00 (6.19)	42.24 (13.33)	36.00 (9.63)	45.20 (15.89)	39.28 (10.20)	53.08 (11.47)	43.52 (8.84)
Diagnosis time [s]	12.16 (20.69)	4.41 (1.99)	35.42 (39.20)	7.67 (3.13)	43.33 (32.54)	19.59 (16.95)	83.55 (76.47)	19.73 (11.09)	116.94 (72.97)	30.16 (16.37)
Replanning time [s]	68.14 (26.28)	63.37 (27.91)	102.53 (34.42)	86.59 (28.20)	137.01 (46.98)	125.11 (48.16)	135.88 (79.00)	117.74 (46.42)	154.78 (66.26)	154.77 (50.21)
Diagnosis accuracy [%]	58.93 (25.43)	82.00 (31.88)	55.92 (27.43)	78.65 (34.53)	45.65 (22.18)	76.00 (31.02)	25.44 (32.10)	66.40 (40.71)	23.00 (28.84)	54.66 (40.40)

TABLE IV
COMPARISON OF REVISED AND RESET APPROACHES FOR DIFFERENT NUMBER OF OBJECTS (COMMONLY SOLVED INSTANCES)

	15 Objects		20 Objects		25 Objects	
	Reset	Revised	Reset	Revised	Reset	Revised
# Replannings	5.05 (2.10)	4.29 (1.26)	6.84 (1.77)	5.69 (1.97)	8.77 (3.19)	5.88 (1.83)
Total plan length	30.05 (5.92)	28.58 (5.55)	39.23 (5.54)	36.76 (6.52)	43.88 (2.84)	36.55 (6.00)
Diagnosis time [s]	9.51 (6.27)	6.65 (2.92)	20.38 (7.91)	13.61 (4.14)	43.51 (14.48)	19.69 (9.22)
Replanning time [s]	73.26 (33.73)	85.38 (30.88)	128.71 (35.13)	118.91 (48.53)	237.95 (67.30)	219.99 (33.79)
Diagnosis accuracy [%]	65.94 (17.50)	85.29 (12.68)	59.94 (13.02)	86.53 (19.40)	52.32 (7.33)	83.33 (17.67)
# Successful instances	17	22	13	21	11	18
# Common successful instances	17		13		9	

observations such that the new hypothesis about the broken robot components is consistent with them, alleviating the disadvantages of the other diagnostic reasoning approaches.

Since the number of successful instances with *augmented diagnosis* is very low, Table II compares *reset diagnosis* and *revised diagnosis* in terms of the number of replannings, the total plan length, and the diagnosis accuracy. The results can be summarized as follows.

- 1) With *revised diagnosis*, generally, the average number of replannings is significantly lower (e.g., 40% lower for instances with five broken parts).
- 2) The scalability of the algorithm in terms of diagnosis time is better with *revised diagnosis* (e.g., 30% lower for instances with five broken parts).
- 3) The diagnosis accuracy is higher for *revised diagnosis* (e.g., 25% higher for instances with two broken parts).

Table II indicates that *reset diagnosis* outperforms *revised diagnosis*, when the number of broken parts is 6. However, this result might be misleading since the number of common successful instances is only 4. Therefore, in Table III, we present the results for all instances even if they do not reach the goal state. The results confirm the conclusions from Table II.

Fig. 8(b) presents the CPU time for the diagnostic reasoning of *revised diagnosis*. As expected, increasing the number of broken parts exponentially increases the diagnosis time.

2) *Scalability Analysis With Respect to the Number of Objects in the Environment*: We have performed a set of experiments in the kitchen domain with four robots, and 15, 20, and 25 objects. We have generated 25 instances where four robotic parts are broken at different times. We have run our algorithm three times for each instance, as described in Section X-A, and reported in Table IV the average and the standard deviation (over 25 instances, 3 runs) of the evaluation metrics described in Section X-B. The results of these experiments are also presented in Fig. 8(c).

Fig. 8(c) presents the success rates of different diagnostic reasoning approaches. According to these results, the execution monitoring algorithm with *revised diagnosis* outperforms all other approaches by a large margin in terms of the success rate.

Table IV compares *reset diagnosis* and *revised diagnosis* in terms of the number of replannings, the total plan length, and the diagnosis accuracy. The results are as follows.

- 1) With *revised diagnosis*, generally, the average number of replannings is significantly lower (e.g., 30% lower for instances with 25 objects).
- 2) The scalability of the algorithm in terms of diagnosis time is better with *revised diagnosis* (e.g., 55% lower for instances with 25 objects).
- 3) The diagnosis accuracy is always higher for *revised diagnosis* (e.g., 30% higher for instances with 25 objects).

XI. CONCLUSION

We introduced a generic plan execution monitoring algorithm based on novel formal methods for prediction, diagnostic reasoning, and replanning. These methods are hybrid in that low-level feasibility checks are embedded in high-level robot domain descriptions.

The prediction method can be used flexibly for different purposes: to detect discrepancies, to check their relevancies, and to infer a full current state for replanning.

The diagnostic reasoning method provides not only the most likely causes for relevant discrepancies but also informative explanations regarding action failures. Our method is general in that it can be utilized in three different modes taking into account the previous observations, previous diagnosis, current observation, and/or earlier observations.

The replanning method is novel in the following ways: it applies under partial observability, it is guided by diagnostic reasoning, and it also considers repairs if needed.

Our execution monitoring algorithm utilizes these methods in a modular way and this modular structure allows the algorithm to be used autonomously as well as interactively to involve the user in decisions and get their feedback.

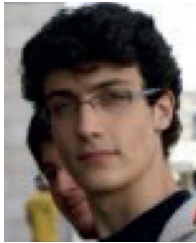
The experimental evaluation results showed the usefulness of diagnostic reasoning with explanations and guided replanning with repairs in plan execution monitoring. They also showed that utilizing the current and the previous observations and being able to revise the previous diagnoses improves the effectiveness.

In this study, we assumed that no other source of information (e.g., by component monitoring or communication) is available about broken parts of robots. Investigation of how such further knowledge can be utilized for diagnostic reasoning is part of our ongoing research.

REFERENCES

- [1] C. Fritz, "Monitoring the generation and execution of optimal plans," Ph.D. dissertation, Univ. Toronto, Toronto, ON, Canada, 2009.
- [2] S. J. Levine, "Monitoring the execution of temporal plans for robotic systems," Ph.D. dissertation, MIT, Cambridge, MA, USA, 2012.
- [3] M. Gelfond and V. Lifschitz, "Classical negation in logic programs and disjunctive databases," *New Gener. Comput.*, vol. 9, pp. 365–385, 1991.
- [4] M. Gelfond and V. Lifschitz, "The stable model semantics for logic programming," in *Proc. Int. Log. Program. Conf. Symp.*, MIT Press, 1988, pp. 1070–1080.
- [5] V. Lifschitz, *Answer Set Program.*. Berlin, Germany: Springer, 2019. [Online]. Available: <https://doi.org/10.1007/978-3-030-24658-7>
- [6] E. Erdem, M. Gelfond, and N. Leone, "Applications of answer set programming," *AI Mag.*, vol. 37, no. 3, pp. 53–68, 2016.
- [7] E. Erdem and V. Patoglu, "Applications of ASP in robotics," *Künstliche Intelligenz*, vol. 32, nos. 2–3, pp. 143–149, 2018.
- [8] G. De Giacomo, R. Reiter, and M. Soutchanski, "Execution monitoring of high-level robot programs," in *Proc. Princ. Knowl. Representation Reasoning*, 1998, pp. 453–465.
- [9] E. Khalastchi and M. Kalech, "On fault detection and diagnosis in robotic systems," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–24, 2018.
- [10] G. Havur, K. Haspalamutgil, C. Palaz, E. Erdem, and V. Patoglu, "A case study on the Tower of Hanoi challenge: Representation, reasoning and execution," in *Proc. of Int. Conf. Robot. Automat.*, 2013, pp. 4552–4559.
- [11] E. Erdem, E. Aker, and V. Patoglu, "Answer set programming for collaborative housekeeping robotics: Representation, reasoning, and execution," *Intell. Serv. Robot.*, vol. 5, pp. 275–291, 2012.
- [12] M. Colledanchise, D. Almeida, and P. Ögren, "Towards blended reactive planning and acting using behavior trees," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 8839–8845.
- [13] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Where's Waldo? Sensor-based temporal logic motion planning," in *Proc. Int. Conf. Robot. Automat.*, 2007, pp. 3116–3121.
- [14] K. He, M. Lahijanian, L. E. Kavraki, and M. Y. Vardi, "Towards manipulation planning with temporal logic specifications," in *Proc. Int. Conf. Robot. Automat.*, 2015, pp. 346–352.
- [15] R. E. Fikes, P. E. Hart, and N. J. Nilsson, "Learning and executing generalized robot plans," *Artif. Intell.*, vol. 3, pp. 251–288, 1972.
- [16] C. Fritz, "Execution Monitoring – A Survey," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2005. [Online]. Available: http://www.cs.toronto.edu/~fritz/publications/depth_oral.pdf
- [17] O. Pettersson, "Execution monitoring in robotics: A survey," *Robot. Auton. Syst.*, vol. 53, no. 2, pp. 73–88, 2005.
- [18] J. A. Ambros-Ingerson and S. Steel, "Integrating planning, execution and monitoring," in *Proc. AAAI Conf. Artif. Intell.*, 1988, pp. 83–88.
- [19] M. Fichtner, A. Großmann, and M. Thielscher, "Intelligent execution monitoring in dynamic environments," *Fundam. Inform.*, vol. 57, nos. 2–4, pp. 371–392, 2003.
- [20] J. Winkler, G. Bartels, L. Mösenlechner, and M. Beetz, "Knowledge enabled high-level task abstraction and execution," in *Proc. Conf. Adv. Cogn. Syst.*, 2012, vol. 2, no. 1, pp. 131–148.
- [21] S. Lemai and F. Ingrand, "Interleaving temporal planning and execution in robotics domains," in *Proc. AAAI Conf. Artif. Intell.*, 2004, pp. 617–622.
- [22] S. Zhang, M. Sridharan, M. Gelfond, and J. Wyatt, "Towards an architecture for knowledge representation and reasoning in robotics," in *Proc. Int. Conf. Social Robot.*, Springer, 2014, pp. 400–410.
- [23] L. P. Kaelbling and T. Lozano-Pérez, "Integrated task and motion planning in belief space," *Int. J. Robot. Res.*, vol. 32, nos. 9–10, pp. 1194–1227, 2013.
- [24] J. P. Mendoza, M. Veloso, and R. Simmons, "Plan execution monitoring through detection of unmet expectations about action outcomes," in *Proc. Int. Conf. Robot. Automat.*, 2015, pp. 3247–3252.
- [25] M. Hanheide et al., "Robot task planning and explanation in open and uncertain worlds," *Artif. Intell.*, vol. 247, pp. 119–150, 2017.
- [26] T. Eiter, E. Erdem, W. Faber, and J. Senko, "A logic-based approach to finding explanations for discrepancies in optimistic plan execution," *Fundam. Inform.*, vol. 79, nos. 1–2, pp. 25–69, 2007.
- [27] R. Reiter, "A theory of diagnosis from first principles," *Artif. Intell.*, vol. 32, no. 1, pp. 57–95, 1987.
- [28] J. De Kleer, A. K. Mackworth, and R. Reiter, "Characterizing diagnoses and systems," *Artif. Intell.*, vol. 56, nos. 2–3, pp. 197–222, 1992.
- [29] N. Roos and C. Witteveen, "Models and methods for plan diagnosis," *Auton. Agents Multi-Agent Syst.*, vol. 19, no. 1, pp. 30–52, 2009.
- [30] S. McIlraith, G. Biswas, D. Clancy, and V. Gupta, "Hybrid systems diagnosis," in *Proc. Int. Workshop Hybrid Syst.: Comput. Control*, Springer, 2000, pp. 282–295.
- [31] M. Balduccini and M. Gelfond, "Diagnostic reasoning with A-Prolog," *Theory Pract. Log. Program.*, vol. 3, nos. 4–5, p. 425–461, 2003.
- [32] C. Baral, S. McIlraith, and T. C. Son, "Formulating diagnostic problem solving using an action language with narratives and sensing," in *Proc. Princ. Knowl. Representation Reasoning*, 2000, pp. 311–322.
- [33] C. Baral, M. Gelfond, and A. Proveti, "Representing actions: Laws, observations and hypotheses," *J. Log. Prog.*, vol. 31, nos. 40, pp. 2–3, 1997.
- [34] V. Marek and M. Truszczyński, "Stable models and an alternative logic programming paradigm," in *The Logic Programming Paradigm: a 25-Year Perspective*. Berlin, Germany: Springer, 1999, pp. 375–398.
- [35] I. Niemelä, "Logic programs with stable model semantics as a constraint programming paradigm," *Ann. Math. Artif. Intell.*, vol. 25, pp. 241–273, 1999.
- [36] V. Lifschitz, "Answer set programming and plan generation," *Artif. Intell.*, vol. 138, pp. 39–54, 2002.
- [37] I. F. Yalciner, A. Nouman, V. Patoglu, and E. Erdem, "Hybrid conditional planning using answer set programming," *Theory Pract. Log. Program.*, vol. 17, nos. 5–6, pp. 1027–1047, 2017.
- [38] M. Rizwan, V. Patoglu, and E. Erdem, "Human robot collaborative assembly planning: An answer set programming approach," *Theory Pract. Log. Program.*, vol. 20, no. 6, pp. 1006–1020, 2020.
- [39] A. Nouman, V. Patoglu, and E. Erdem, "Hybrid conditional planning for robotic applications," *Int. J. Robot. Res.*, vol. 40, no. 2–3, pp. 594–623, 2020.
- [40] D. Das, S. Banerjee, and S. Chernova, "Explainable AI for robot failures: Generating explanations that improve user assistance in fault recovery," in *Proc. Conf. Hum.-Robot Interact.*, 2021, pp. 351–360.
- [41] D. E. Smith, "Planning as an iterative process," in *Proc. AAAI Conf. Artif. Intell.*, J. Hoffmann and B. Selman, Eds., 2012, pp. 2180–2185.

- [42] T. Chakraborti, S. Sreedharan, and S. Kambhampati, "The emerging landscape of explainable automated planning & decision making," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 4803–4811.
- [43] E. Erdem, V. Patoglu, and Z. G. Saribatur, "Integrating hybrid diagnostic reasoning in plan execution monitoring for cognitive factories with multiple robots," in *Proc. Int. Conf. Robot. Automat.*, 2015, pp. 2007–2013.
- [44] J. McCarthy, "Elaboration tolerance," in *Proc. Int. Symp. Commonsense Reasoning*, 1998, pp. 198–216.
- [45] T. Eiter, G. Ianni, R. Schindlauer, and H. Tompits, "A uniform integration of higher-order reasoning and external evaluations in answer-set programming," in *Proc. Int. Joint Conf. Artif. Intell.*, 2005, pp. 90–96.
- [46] C. Dornhege, P. Eyerich, T. Keller, S. Trüg, M. Brenner, and B. Nebel, "Semantic attachments for domain-independent planning systems," in *Proc. 19th Int. Conf. Automated Plan. Scheduling*, 2009, pp. 114–121.
- [47] S. Thiébaux, J. Hoffmann, and B. Nebel, "In defense of PDDL axioms," *Artif. Intell.*, vol. 168, nos. 1–2, pp. 38–69, 2005.
- [48] G. Brewka, T. Eiter, and M. Truszczynski, "Answer set programming at a glance," *Commun. ACM*, vol. 54, no. 12, pp. 92–103, 2011.
- [49] M. Gebser, B. Kaufmann, R. Kaminski, M. Ostrowski, T. Schaub, and M. T. Schneider, "Potassco: The Potsdam answer set solving collection," *AI Commun.*, vol. 24, no. 2, pp. 107–124, 2011.
- [50] F. Buccafurri, N. Leone, and P. Rullo, "Enhancing disjunctive datalog by constraints," *IEEE Trans. Knowl. Data Eng.*, vol. 12, no. 5, pp. 845–860, Sep/Oct. 2000.
- [51] E. Erdem, V. Patoglu, and P. Schüller, "A systematic analysis of levels of integration between high-level task planning and low-level feasibility checks," *AI Commun.*, vol. 29, no. 2, pp. 319–349, 2016.
- [52] Z. G. Saribatur, V. Patoglu, and E. Erdem, "Finding optimal feasible global plans for multiple teams of heterogeneous robots using hybrid reasoning: An application to cognitive factories," *Auton. Robots*, vol. 43, pp. 213–238, 2019.
- [53] R. Diankov, "Automated construction of robotic manipulation programs," Ph.D. dissertation, Carnegie Mellon Univ., Robotics Institute, Aug. 2010.



Gokay Coruhlu received the B.S. and M.S. degrees in mechatronics engineering in 2012 and 2014, respectively, from Sabanci University, Istanbul, Turkey, where he is currently working toward the Ph.D. degree in mechatronics engineering.

His research interests include control of haptic devices, rehabilitation robotics, human–robot interaction, execution monitoring, and diagnostic reasoning.



Esra Erdem (Member, IEEE) received the Ph.D. degree in computer sciences from the University of Texas at Austin, Austin, TX, USA, in 2002.

She is a Professor in Computer Science and Engineering with Sabanci University, Istanbul, Turkey. She carried out postdoctoral research at the University of Toronto, Toronto, ON, Canada, and Vienna University of Technology, Vienna, Austria, from 2002 to 2006. Her research is in the area of artificial intelligence, in particular, the mathematical foundations of knowledge representation and reasoning and their

applications to cognitive robotics and computational biology.

Dr. Erdem is an Associate Editor for the *Journal of Artificial Intelligence (AIJ)* and *Theory and Practice of Logic Programming (TPLP)*.



Volkan Patoglu (Member, IEEE) received the Ph.D. degree in mechanical engineering from the University of Michigan, Ann Arbor, MI, USA, in 2005.

He is a Professor of Mechatronics Engineering with Sabanci University, Istanbul, Turkey. He worked as a Postdoctoral Research Associate with Rice University, Houston, TX, USA, in 2006. His research is in the area of physical human–machine interaction, in particular, design and control of force feedback robotic systems with applications to rehabilitation. His research extends to cognitive robotics.

Dr. Patoglu has served as an Associate Editor for IEEE TRANSACTIONS ON HAPTICS from 2013 to 2017 and is currently an Associate Editor for IEEE TRANSACTIONS ON NEURAL SYSTEMS AND REHABILITATION ENGINEERING and *IEEE Robotics and Automation Letters*.